

Advanced Machine Learning

Non scholae, sed vitae discimus

Vincenzo Salvatore

Raffaele Mercurio





Contents

1 Foundations of Deep Learning and Optimization 7

1.1 Introduction to the Core Concepts 7

1.2 Parametric Models and Optimization Foundations 8

1.2.1 Linear Regression ($f : \mathbb{R}^D \to \mathbb{R}$) 8

1.2.2 Logistic Regression ($f : \mathbb{R}^D \to [0, 1]$) 8

1.2.3 Softmax Classifier ($f : \mathbb{R}^D \to \{0, \ldots, K\}$) 9

1.3 Deep Feedforward Networks and Backpropagation 11

1.3.1 Deep Neural Networks 11

1.3.2 Backpropagation 11

1.4 The Role of Activation Functions 15

1.4.1 ReLU (Rectified Linear Unit) 15

1.4.2 Leaky ReLU 15

1.4.3 ELU (Exponential Linear Units) 16

1.4.4 GELU (Gaussian Error Linear Unit) 16

1.5 Weight Initialization and Signal Propagation 17
1.5.1 The Pitfalls of Naive Initialization . . . 17
1.5.2 Variance Scaling Methods: The Solution . . . 17
1.6 Gradient-Based Optimization Algorithms 18
1.7 Regularization in Deep Neural Networks 19
1.7.1 Dropout . . . 19
1.7.2 The Common Pattern of Modern Regularization . . . 20
1.8 Feature Normalization Techniques 20
1.8.1 Batch Normalization . . . 21
1.8.2 Alternative Normalization Methods . . . 21
1.9 Chapter Summary and Conceptual Graph 22

2 Convolutional Neural Networks and Computer Vision . . . 25
2.1 Image Representation and the Imaging Process 25
2.2 Motivations for Convolutions 26
2.2.1 The Problem with Fully Connected Layers . . . 26
2.2.2 Exploiting Image Structure: Locality and Stationarity . . . 26
2.3 The Core Building Block: The Convolutional Layer 26
2.3.1 Key Hyperparameters of the Convolution . . . 27
2.3.2 Formal Mathematical Definition . . . 28
2.4 Classic Image Processing Filters 28
2.5 Output Shape and Parameter Counting for 3D Volumes 30
2.5.1 Spatial Output Size Formula . . . 30
2.6 The Pooling Layer 30
2.6.1 Purpose and Properties of Pooling . . . 30
2.6.2 Max Pooling . . . 30
2.6.3 Average Pooling . . . 31
2.7 Visualizing Convolutional Networks 31
2.8 Vision Regularization: Data Augmentation 31
2.8.1 Classic Spatial and Color Augmentations . . . 32
2.8.2 Advanced Stochastic Augmentations . . . 32
2.9 Transfer Learning in Computer Vision 33
2.9.1 The Fundamental Concept (Why Use It?) . . . 33
2.9.2 Practical Implementation . . . 33
2.9.3 Pervasiveness: The Modern Standard . . . 33
2.10 Chapter Summary and Conceptual Graph 34

3 Sequence Models 37

3.1 Introduction to Sequence Modeling 37
3.1.1 Taxonomy of Sequence Tasks 38
3.1.2 Forecasting as a Core Sequence Problem 39

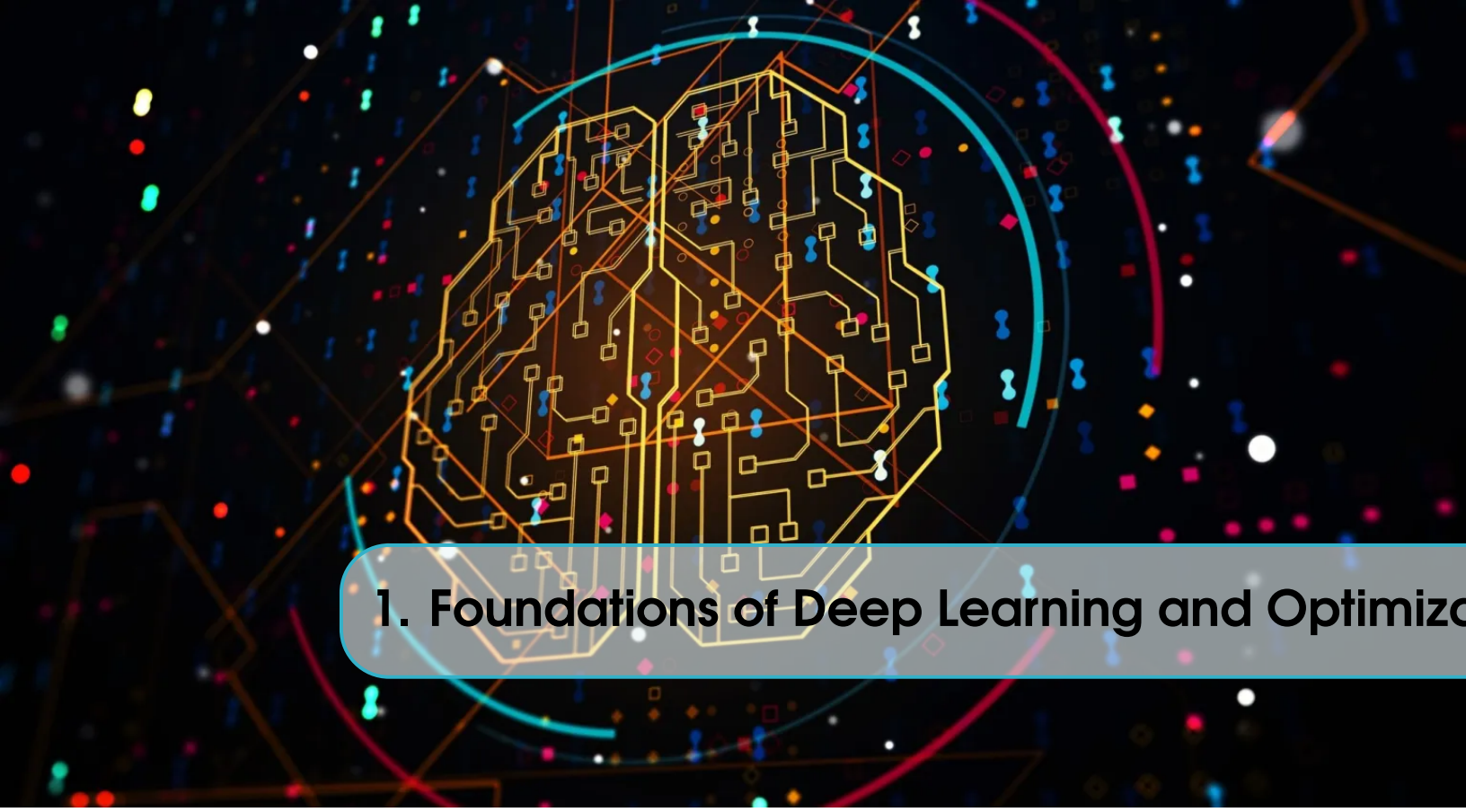
3.2 Temporal Convolutional Networks 39
3.2.1 One-Dimensional Convolutions on Sequences 39
3.2.2 Causal and Dilated Convolutions 40
3.2.3 Trade-offs of Temporal Convolutional Networks 42

3.3 Recurrent Neural Networks 43
3.3.1 The Recurrence Principle 43
3.3.2 The Vanilla RNN Formulation 43
3.3.3 Unrolling the Computational Graph 44
3.3.4 Sequence-to-Sequence: Synchronized vs. Encoder-Decoder Mappings 44
3.3.5 Case Study: Character-Level Language Modeling 46
3.3.6 Case Study: Image Captioning (CNN + RNN) 48

3.4 Backpropagation Through Time and Gradient Flow 49
3.4.1 BPTT and Gradient Flow in Vanilla RNNs 49
3.4.2 Exploding and Vanishing Gradients 50
3.4.3 Vanilla RNN Trade-offs 50

3.5 Long Short-Term Memory 51
3.5.1 The Dual-State Architecture 51
3.5.2 The LSTM Cell Architecture 51
3.5.3 Detailed Breakdown of Cell Components 52
3.5.4 Why LSTM Solves the Vanishing Gradient Problem 53

3.6 Chapter Summary and Conceptual Graph 54



1. Foundations of Deep Learning and Optimiza

1.1 Introduction to the Core Concepts

The contents of this chapter establish the theoretical and mathematical foundations of modern deep learning, largely based on the introductory modules of the course [1].

Key Word

- According to Arthur Samuel (1959), **Machine Learning** is the field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) defined the well-posed learning problem: a computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

Machine learning is heavily applied in **Computer Vision**, which can be viewed from three main perspectives: as a science (exploring the computational model of human perception), as engineering (building systems that perceive the world), and through its applications.

Example

Some typical **Computer Vision Applications** include:

- **Medical imaging**: Supporting medical diagnosis and visualization.
- **Surveillance**: Tracking people in airports or train stations.
- **Autonomous driving**: Lane-keeping and pre-crash intervention.

—

1.2 Parametric Models and Optimization Foundations

The majority of machine learning models can be formulated as **Parametric Models**. A parametric model takes an input x and uses a set of parameters (or weights) W and biases b to produce an output prediction z:

$$z = f(x, W) = Wx + b$$

While the exact nature of the prediction depends on the task, what unifies these models is the **Optimization Process**. We define a Loss function L that measures the error of our predictions, and we iteratively adjust the weights using **Gradient Descent**:

$$W^{t+1} = W^t - \alpha \nabla_W L$$

where α is the learning rate.

By simply changing the *activation function* applied to the linear score z and the corresponding *loss function*, we obtain different classical models: Linear Regression, Logistic Regression, and the Softmax Classifier.

1.2.1 Linear Regression ($f : \mathbb{R}^D \to \mathbb{R}$)

Linear Regression is used when the target variable is continuous. We use the identity activation function (i.e., no activation), so the predicted output $\hat{y}$ is exactly the linear score:

$$\hat{y} = z = Wx + b$$

We optimize this model by minimizing the **Mean Squared Error (MSE)** loss:

$$L = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

1.2.2 Logistic Regression ($f : \mathbb{R}^D \to [0, 1]$)

Logistic Regression is used for binary classification. We map the linear score z to a probability between 0 and 1 using the **Sigmoid** activation function $\sigma(z)$:

$$P(Y = 1 | X = x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The appropriate loss function for this model is the **Binary Cross-Entropy Loss (BCE)**. For a dataset of N samples, it is defined as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[\underbrace{y_i \log(\sigma(z_i))}_{\text{Active if } y=1} + \underbrace{(1-y_i)\log(1-\sigma(z_i))}_{\text{Active if } y=0}\right]$$

Note

Understanding the BCE Formula: This equation uses a clever mathematical trick to combine two mutually exclusive cases into a single line:

- When the true class is $y_i = 1$, the second term $(1-1) = 0$ disappears, and the loss evaluates only $-\log(\sigma(z_i))$, penalizing the model if it predicts a low probability.
- When the true class is $y_i = 0$, the first term 0 disappears, and the loss evaluates only $-\log(1-\sigma(z_i))$, penalizing the model if it predicts a high probability for class 1.

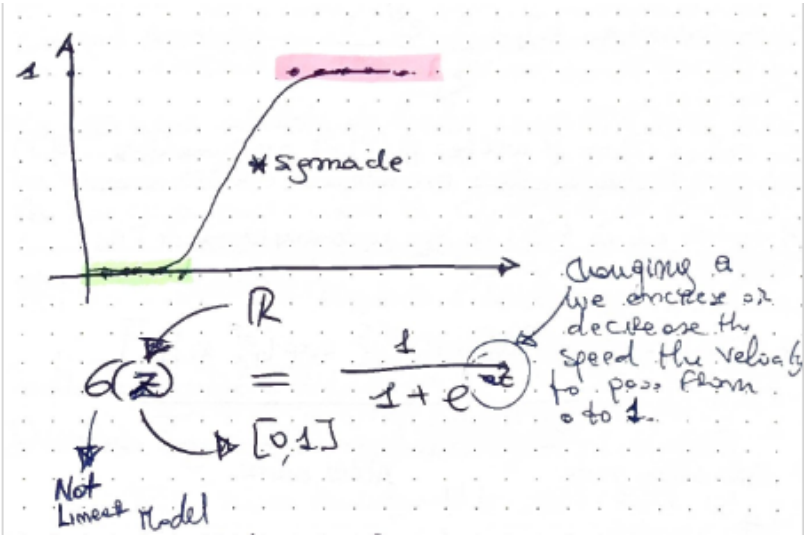


Figure 1.1: Illustration of the Sigmoid function in Logistic Regression.

1.2.3 Softmax Classifier ($f : \mathbb{R}^D \to \{0,\ldots,K\}$)

The Softmax Classifier generalizes Logistic Regression to multiple, mutually exclusive classes. It maps the raw scores z into a valid probability distribution over the classes using the **Softmax function**:

$$P(Y = k|X = x_i) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Note

Why Softmax? We want to interpret raw classifier scores as *probabilities*. The Softmax function transforms scores such that they are strictly positive and sum to 1.

Example

For an input image of size $32 \times 32 \times 3$ (e.g., CIFAR-10), the flattened array x has 3072 numbers. If we are classifying into 10 categories, the weight matrix W will have dimensions 10×3072, producing a 10×1 vector of raw class scores before the Softmax is applied.

We then optimize the model using the **Categorical Cross-Entropy Loss**, often referred to as the Negative Log-Likelihood (NLL). Over the entire dataset of N samples, expanding the Softmax probability formula, the loss becomes:

$$NLL(W,X) = -\sum_{i=1}^{N} \left[\underbrace{W_{y_i}^T x_i}_{\text{Punteggio della classe giusta}} - \underbrace{\log\left(\sum_{k=1}^{K} \exp(W_k^T x_i)\right)}_{\text{Somma dei punteggi delle altre classi}} \right]$$

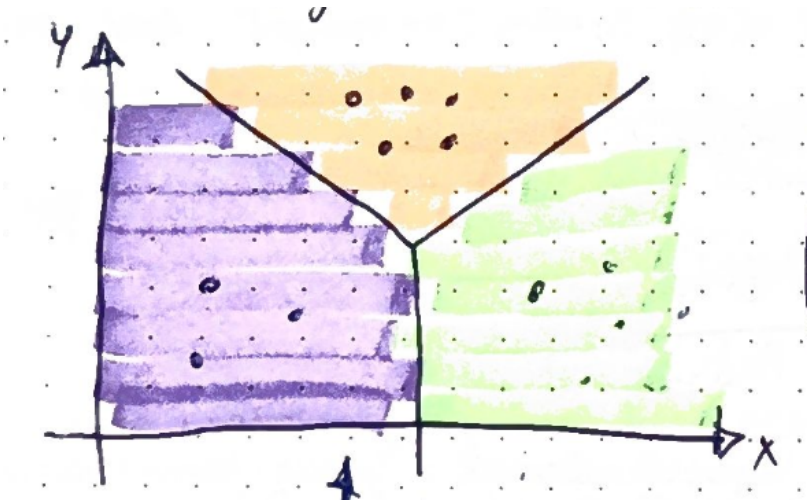


Figure 1.2: Abstract representation of logistic regression.

Note

Understanding the Softmax NLL Formula: By looking at the expanded formula, we see that the optimizer tries to maximize the linear score $W_{y_i}^T x_i$ of the *correct class* y_i, while simultaneously pushing down the log-sum of the scores of *all available classes*. This creates a competitive environment where increasing the score of the correct class implicitly decreases the relative probability of the others.

—

1.3 Deep Feedforward Networks and Backpropagation

1.3.1 Deep Neural Networks

By stacking multiple layers and applying non-linear activation functions, we construct multi-layer Neural Networks (Multi-Layer Perceptrons, MLPs):

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

This hierarchical representation enables the network to learn rich, non-linear representations from raw data.

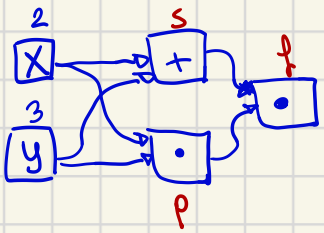
1.3.2 Backpropagation

To train a deep network, we compute the gradient of the loss with respect to all parameters using the **Backpropagation** algorithm, which systematically applies the multivariable chain rule from the output back to the inputs (backward pass).

Note

Backpropagation with Matrices: When inputs and weights are vectors or matrices, local gradients become *Jacobian matrices*. However, Jacobians are often sparse. Instead of explicitly forming them, practical frameworks use implicit vector-Jacobian products. A crucial dimensional consistency check to remember: the downstream gradient $\frac{\partial L}{\partial x}$ **always has the exact same shape** as the tensor x!

Here are some solved exercises demonstrating the Forward and Backward pass step-by-step:



FORWARD PASS

$s = x+y = 2+3 = 5$

$p = x \cdot y = 2 \cdot 3 = 6$

$f = s \cdot p = 30$

Reverse Pass

- $f = s \cdot p$ → $\frac{\partial f}{\partial s} = p$, $\frac{\partial f}{\partial p} = s$
- $p = x \cdot y$ → $\frac{\partial p}{\partial x} = y$, $\frac{\partial p}{\partial y} = x$
- $s = x+y$ → $\frac{\partial s}{\partial x} = 1+0=1$, $\frac{\partial s}{\partial y} = 0+1=1$

Applico la chain Rule

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial x} + \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial x} = p \cdot 1 + s \cdot y = 6 \cdot 1 + 5 \cdot 3 = 21$$

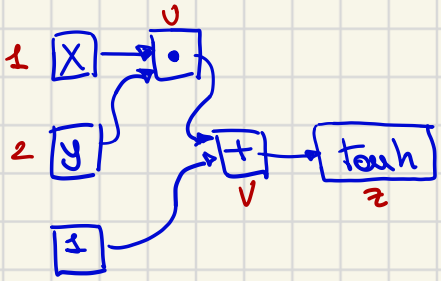
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial y} + \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial y} = p \cdot 1 + s \cdot x = 6 \cdot 1 + 5 \cdot 2 = 16$$

Cosa capisco da questo esercizio? Capisco quanto un Peso x e y impattano rispetto alla funzione di Loss f. Vedo che X ha un impatto maggiore.

Esercizio 3

Consider a 2-Layer function: $z = \tanh(xy+1)$ $x=1$ e $y=2$

- Build a computation graph:
- forward pass: compute z.
- Reverse pass: compute gradient $\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y}$.



FORWARD:

- $v = x \cdot y = 2$
- $v = v+1 = 3$
- $z = \tanh(v) = \tanh(3) \simeq 0{,}995$

Reverse PASS

- $z = \tanh(v) \rightsquigarrow \frac{\partial z}{\partial v} = \operatorname{sech}^2(v) = 1 - \tanh^2(v) \simeq 1 - (0.995)^2 \simeq 0.01$
- $v = v+1 = \rightsquigarrow \frac{\partial v}{\partial v} = 1$
- $v = x \cdot y \rightsquigarrow \frac{\partial v}{\partial x} = y = 2$, $\frac{\partial v}{\partial y} = x = 1$
- Chain Rule

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial v} \cdot \frac{\partial v}{\partial x} = 0{,}01 \cdot 1 \cdot 2 \simeq 0{,}02$$

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial v} \cdot \frac{\partial v}{\partial y} = 0{,}01 \cdot 1 \cdot 1 \simeq 0{,}01$$

Esercizio 4

given: $g(w) = (w-3)^2$ $w_0 = 0$ $\alpha = 0{,}1$

$\boxed{g'(w) = 2(w-3)}$

$$w^k = w^{k-1} - \alpha \nabla g(w^{k-1})$$

① WRITE the gradient descent Rule
② Compute the first 5 iter... $w_1 \dots w_5$
③ sKetch the cost function values.

$w_{k+1} = w_k - \alpha g(w_k) =$
$w_{k+1} = w_k - 0,2(w_k - 3) =$
$w_{k+1} = w_k - 0,2 w_k + 0,6$
$w_{k+1} = (1-0,2) w_k + 0,6$
$w_{k+1} = 0,8 w_k + 0,6$

Notiamo come solo lo 0,8% del vecchio valore è mantenuto

$w_0 = 0$
$w_1 = 0,8 \cdot 0 + 0,6 = 0,6$
$w_2 = 0,8 \cdot 0,6 + 0,6 = 1,08$
$w_3 = 0,8 \cdot 1,08 + 0,6 = 1,464$
$w_4 = 0,8 \cdot 1,464 + 0,6 = 1,7712$
$w_5 = 0,8 \cdot 1,7712 + 0,6 = 2,01696$

Ad ogni passo l'iterazione ci spinge un po più vicino al minimo reso $w^* = 3$. La velocità con cui ci avviciniamo a 3 diminuisce gradualmente.

Esercizio 5 $g(w_1, w_2) = w_1^2 + 2w_2^2$

① Compute the gradient $\nabla g(w)$
② Apply GD from initial point $(w_1, w_2) = (2, 1)$ with learning rate $\alpha = 0,2$
③ Compute the first 3 update explicitly.

for computing the gradient $\nabla g(w_1, w_2)$ we have to apply the first derivate to the function $g(w_1, w_2) \rightsquigarrow g(w_1, w_2) = 2w_1 + 4w_2$

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 4w_2 \end{bmatrix}$$ ← Il gradiente è la derivata della funzione.

Rule for each weight:

w_1
$w_1^{k+1} = w_1^k - \alpha \nabla g(w_1) =$
$w_1^{k+1} = w_1^k - 0,2(2w_1^k) =$
$w_1^{k+1} = w_1^k - 0,4 w_1^k =$
$\boxed{w_1^{k+1} = 0,6 w_1^k}$

w_2
$w_2^{k+1} = w_2^k - \alpha \nabla g(w_2^k) =$
$w_2^{k+1} = w_2^k - 0,2(4w_2^k)$
$w_2^{k+1} = w_2^k - 0,8 w_2^k =$
$\boxed{w_2^{k+1} = 0,2 w_2^k}$

Questo è il modo in cui cambieremo i Pesi.

Let's compute the iteration starting with $(2, 1)$
$K=1 \rightsquigarrow (w_1, w_2) = (0.6 \cdot 2, 0.2 \cdot 1) = (1.2, 0.2)$
$K=2 \rightsquigarrow (1.2, 0.2) = (0,72, 0,04)$
$K=3 \rightsquigarrow (0,72, 0.04) = (0.432, 0.008)$

Obiettivo: Vogliamo capire come la funzione di LOSS guida l'aggiornamento dei Pesi per far sì che la nostra Rete produca output sempre più vicini a quelli desiderati.

EX.

Immagina di avere un modello lineare usiamo ad esempio $(X, Y) = (2, 5)$

$\hat{y} = w \cdot X + b$ (peso, bias, input, output predetto)

PASSO 1: Inizializzo i PESI

Pesi iniziali: $w = 0,5$ $b = 0,0$

! noi non sappiamo nulla quindi impostiamo valori casuali.

PASSO 2: FORWARD PASS (PREDIZIONE)

$\hat{y} = w \cdot x + b = 0.5 \cdot 2 + 0 = 1$ ← PREDIZIONE

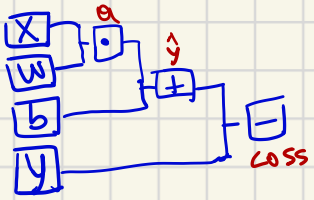
PASSO 3: calcolo della LOSS (ERRORE). usiamo MSE

$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{(1-5)^2}{2} = \frac{16}{2} = 8$ ← siamo molto lontani dal TARGET.

PASSO 4: Backpropagation (calcolo del GRADIENTE): come cambiamo i Pesi ??

❗ Deriviamo la Loss Rispetto a $\hat{y}$

GRAFO COMPUT:



$\begin{cases} a = w \cdot x \\ \hat{y} = a + b \\ L = \hat{y} - y \end{cases}$

BACKWARD PASS

- $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$
- $\frac{\partial \hat{y}}{\partial a} = 1$
- $\frac{\partial a}{\partial w} = x$
- $\frac{\partial \hat{y}}{\partial b} = 1$

w ↦ $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial w} = (\hat{y} - y) \cdot x = (-4) \cdot 2 = \boxed{-8}$

b ↦ $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = (\hat{y} - y) \cdot 1 = \boxed{-4}$

PASSO 5: GD update dei Pesi. Regola di aggiornamento: $w := w - \alpha \frac{\partial L}{\partial w}$

$b := b - \alpha \frac{\partial L}{\partial b}$

$\alpha = 0{,}1$

$w^1 = 0.5 - 0.1 \cdot (-8) = 1{,}3$ NUOVI PESI K=1

$b^1 = 0 - 0.1 \cdot (-4) = 0{,}4$

PREDICT

$\hat{y} = w' x + b' = 1{,}3 \cdot 2 + 0{,}4 = 3$ Nota come la predizione diventa più precisa e vicino al 5 del DATASET.

Ovviamente questo Processo viene eseguito su ogni Riga del Dataset. Non sulla stessa.

—

1.4 The Role of Activation Functions

An **activation function** is a mathematical operator applied to the affine output of a layer ($z = Wx + b$) before passing it to subsequent layers. Its primary role is to introduce **non-linearity** into the network.

Note

Why do we need Non-Linearity? If we only stacked linear layers without activation functions, the entire deep network would mathematically collapse into a single linear transformation. For example, a 2-layer linear network $f = W_2(W_1 x)$ can be rewritten as $f = W'x$ where $W' = W_2 W_1$. Non-linear activations allow the network to approximate arbitrarily complex, non-linear decision boundaries.

Some of the most widely used activation functions are illustrated in Figure 1.3:

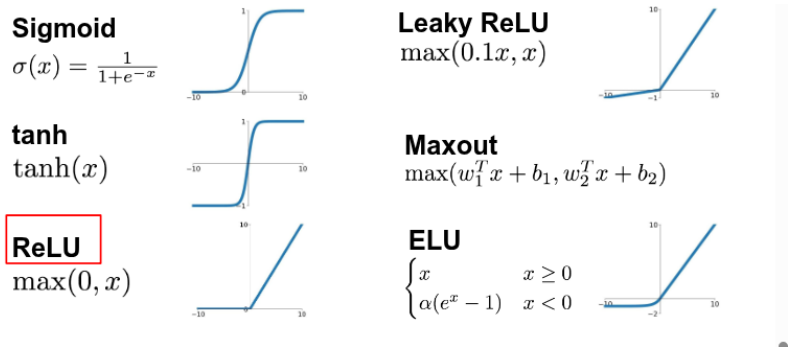


Figure 1.3: Illustration of common activation functions.

1.4.1 ReLU (Rectified Linear Unit)

Formula: $f(x) = \max(0, x)$

ReLU is the classic default activation function across deep learning architectures.

- **Pros:** It does not saturate in the positive region. It is highly computationally efficient (just a thresholding operation) and converges significantly faster than Sigmoid or Tanh in practice (often roughly $6\times$ faster).
- **Cons:** Its outputs are not zero-centered. Most importantly, it suffers from the **"Dead ReLU"** problem: if an activation falls into the negative regime ($x < 0$), its local gradient is exactly 0. If a neuron is pushed into this regime by an unfortunate large weight update, it will never update its weights again.

1.4.2 Leaky ReLU

Formula: $f(x) = \max(\alpha x, x)$ (where typically $\alpha = 0.01$)

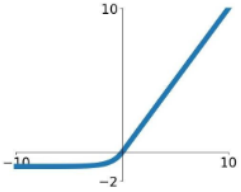
Designed specifically to prevent the "dead neuron" problem of standard ReLU.

- **Pros:** Retains all the computational advantages of ReLU (no positive saturation, fast evaluation), while the small non-zero slope for negative inputs guarantees that neurons will never permanently die.

1.4.3 ELU (Exponential Linear Units)

Formula: $f(x) = x$ if $x > 0$, else $\alpha(e^x - 1)$

- **Pros:** Inherits the benefits of ReLU, but produces outputs with a mean closer to zero. Its smooth negative saturation regime adds noise robustness compared to unbounded negative regimes.
- **Cons:** More computationally demanding because evaluating the exponential function e^x is slower than simple comparisons.



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

Figure 1.4: Illustration of ELU.

1.4.4 GELU (Gaussian Error Linear Unit)

Formula: $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution.

- **Pros:** Exhibits smooth probabilistic gating behavior around 0. Its continuous curvature facilitates optimization in state-of-the-art architectures, making it the standard choice in **GPT, BERT**, and modern Vision Transformers.
- **Cons:** Slightly higher computational cost than ReLU.

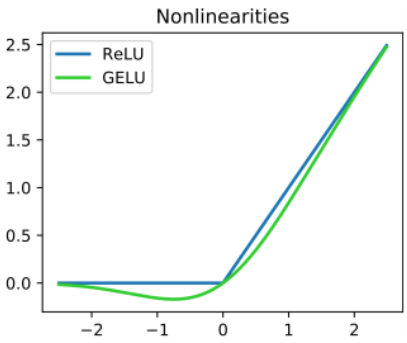


Figure 1.5: Illustration of GELU.

Note

TL;DR — Practical Guidelines for Activation Functions:

- Use **ReLU** as your standard baseline, monitoring learning rates to avoid dead neurons.
- Explore **Leaky ReLU** or **GELU** for improved gradient flow and smoother optimization.
- Avoid **Sigmoid/Tanh** for intermediate hidden layers of deep networks, reserving them strictly for output gates or bounded representations (such as in LSTM cells).

—

1.5 Weight Initialization and Signal Propagation

Properly initializing the weights of a neural network is crucial. With poor initialization, training can fail entirely before it starts due to signal collapse or numerical explosion.

1.5.1 The Pitfalls of Naive Initialization

- **Constant Initialization ($W =$ constant):** If all weights are initialized to the same value (e.g., $W = 0$), all neurons in a given layer will produce identical outputs and receive identical gradients during backpropagation. The network fails to *break symmetry* and cannot learn independent features.

Small Random Noise and Vanishing Gradients

A common naive approach is initializing weights with small random numbers, such as `W = 0.01 * np.random.randn(D, H)`, where weights are sampled from a standard normal distribution scaled by a tiny standard deviation of 0.01.

In deep networks, this naive scaling causes complete signal extinction:

- **Step 1: The Forward Pass (Shrinking Activations).**
  The histograms in Figure 1.6 illustrate the distribution of **activations** across successive layers. Because the weights are scaled by 0.01, the output of each layer is strictly smaller in variance than its input. Across sequential layers, this repeated multiplication causes the signal energy to exponentially decay. By Layer 6, activations have completely collapsed to 0 (std $= 0.05$).
- **Step 2: The Backward Pass (Vanishing Gradients).**
  During backpropagation through a linear layer $y = Wx$, the local gradient with respect to weights is $\frac{\partial y}{\partial W} = x$. Therefore, the gradient of the Loss with respect to W is:

$$\frac{\partial L}{\partial W} = x^T \frac{\partial L}{\partial y}$$

The weight update is directly proportional to the magnitude of the input activations x. Because the forward pass collapsed the activations to 0, the weight gradient evaluates to $\frac{\partial L}{\partial W} \approx 0 \cdot \frac{\partial L}{\partial y} = 0$, bringing learning to a complete standstill.

1.5.2 Variance Scaling Methods: The Solution

To guarantee that signal variance remains stable across deep layers, we scale initial weights **dynamically** based on the fan-in (D_{in}) of each neuron:

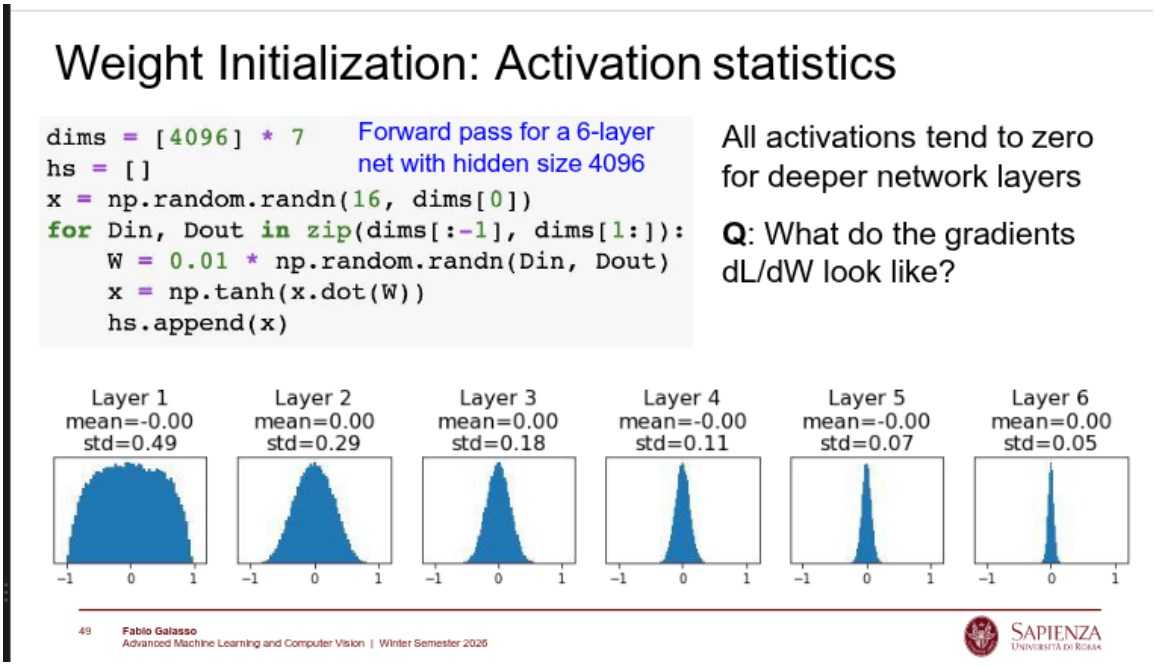


Figure 1.6: Distribution of activations x across 6 layers with $W \sim \mathcal{N}(0, 0.01)$ and Tanh.

Key Word

1. Xavier (Glorot) Initialization

Designed for symmetric, zero-centered activation functions like **Tanh** or **Sigmoid**. The variance of activations is kept constant across layers by drawing weights from a distribution with standard deviation:

$$\text{std} = \frac{1}{\sqrt{D_{in}}}$$

2. Kaiming (He) Initialization

Xavier initialization fails with **ReLU** because ReLU zeroes out all negative values, effectively halving the variance at each step. Kaiming initialization compensates by doubling the variance, setting the standard deviation to:

$$\text{std} = \sqrt{\frac{2}{D_{in}}}$$

This is the standard and mandatory initialization when training deep ReLU networks.

—

1.6 Gradient-Based Optimization Algorithms

From a theoretical standpoint, our objective is to minimize the total loss over the entire dataset:

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

The exact gradient requires summing over all N training examples:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Computing this **Full Batch** gradient requires evaluating millions of samples at every parameter update, which is computationally prohibitive.

Key Word

Stochastic Gradient Descent (SGD): Instead of the full sum, SGD approximates the true gradient over a small random **minibatch** of examples (e.g., 32, 64, or 128). While this introduces stochastic variance, it allows frequent, computationally cheap updates that accelerate convergence.

—

1.7 Regularization in Deep Neural Networks

The objective of **Regularization** is to prevent models from overfitting training data by controlling model capacity. We add a parameter complexity penalty $R(W)$ to the empirical loss:

$$L_{total}(W,X) = \underbrace{L(W,X)}_{\text{Data Loss}} + \underbrace{\lambda R(W)}_{\text{Regularization Term}}$$

Note

Role of the Hyperparameter λ:
- $\lambda = 0$: The model focuses solely on fitting training data, risking **Overfitting**.
- $\lambda \to \infty$: The penalty dominates, forcing all weights toward zero and causing **Underfitting**.

Classic mathematical penalties include:

- **L2 Regularization (Weight Decay / Ridge):** $R(W) = \sum W^2$. Penalizes large weights quadratically, encouraging diffuse representations across all neurons.
- **L1 Regularization (Lasso):** $R(W) = \sum |W|$. Drives less relevant weights exactly to zero, performing automatic feature selection.

1.7.1 Dropout

Introduced by Srivastava et al. (2014), **Dropout** is a stochastic regularization technique where individual neurons are randomly deactivated during training with probability p (commonly $p = 0.5$).

- **Prevents Co-adaptation:** Neurons cannot rely on the specific activations of neighboring units, forcing them to learn robust, self-reliant features.
- **Ensemble Effect:** Dropout implicitly trains an exponential ensemble of shared-weight subnetworks (2^N sub-architectures for an N-neuron layer).

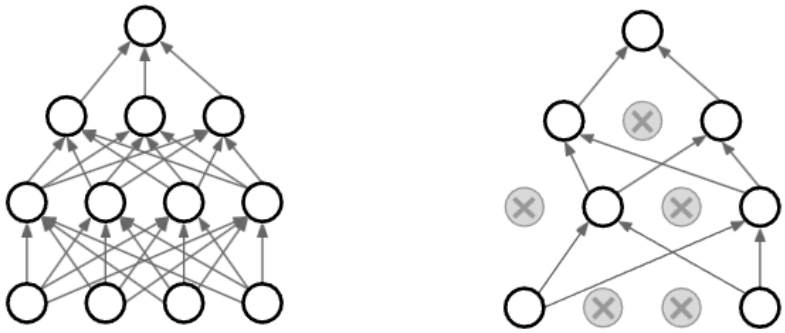


Figure 1.7: Illustration of Dropout during training.

Inverted Dropout

Because dropping neurons during training reduces the active sum of signals, the expected output at test time (when all neurons are active) would be scaled by $\frac{1}{1-p}$. To eliminate any scaling overhead at inference time, modern implementations use **Inverted Dropout** during training:

$$h_{\text{train}} = \frac{\text{mask} \odot x}{1-p}$$

- **Numerator (mask $\odot x$):** Drops a random fraction p of the activations to zero.
- **Denominator ($1-p$):** Inflates the surviving activations, ensuring that the expected activation value during training matches the deterministic magnitude at test time ($h_{\text{test}} = x$).

Example

Numerical Example of Inverted Dropout ($p = 0.5$):
Given input $x = [10, 20, 30, 40]$ (Sum $= 100$) and mask $[1, 0, 1, 0]$:

$$\text{Masked output: } [10, 0, 30, 0] \quad (\text{Sum drops to } 40)$$

$$h_{\text{train}} = \frac{[10, 0, 30, 0]}{0.5} = [20, 0, 60, 0] \quad (\text{Scaled sum } = 80 \approx 100)$$

1.7.2 The Common Pattern of Modern Regularization

Dropout illustrates the universal recipe of modern deep learning regularization:

1. **Training:** Inject stochasticity or random noise.
2. **Testing:** Marginalize or average out the randomness to yield deterministic predictions.

—

1.8 Feature Normalization Techniques

To stabilize signal distributions and accelerate optimization in deep networks, activations are explicitly normalized across intermediate layers.

1.8.1 Batch Normalization

Batch Normalization (BN) normalizes activations across the minibatch dimension, forcing each feature channel k to zero mean and unit variance:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}] + \varepsilon}}$$

where $\mathrm{E}[x^{(k)}]$ and $\mathrm{Var}[x^{(k)}]$ are the minibatch mean and variance, and ε prevents division by zero.

Learnable Scale and Shift Parameters

To prevent rigid normalization from limiting network capacity, two learnable parameters (γ and β) restore representational flexibility:

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}$$

If beneficial, the optimizer can recover the identity mapping by setting $\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$ and $\beta^{(k)} = \mathrm{E}[x^{(k)}]$.

Training versus Test Phase

During training, statistics are calculated over the current minibatch. During inference, fixed global running averages computed during training are used, allowing BN to be mathematically folded into preceding linear layers with zero inference overhead.

1.8.2 Alternative Normalization Methods

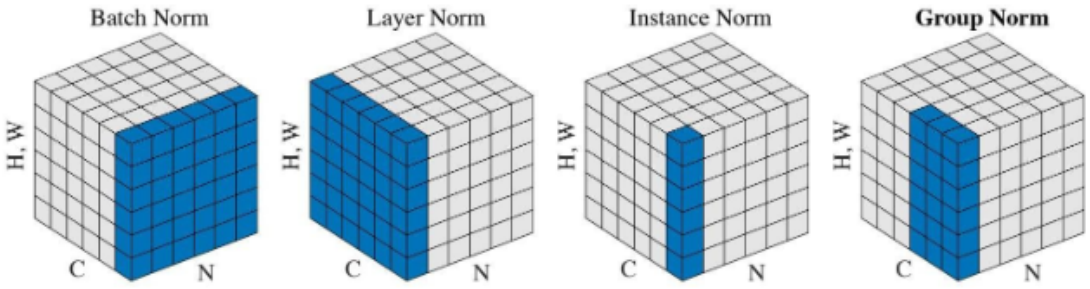


Figure 1.8: Comparison of different normalization methods across batch (N), channel (C), and spatial (H, W) dimensions.

The choice among normalization methods depends on the axes across which statistics are aggregated (Figure 1.8):

Batch Norm (BN)

Aggregates across the batch dimension N and spatial dimensions (H, W) independently for each channel C. Ideal for vision models with large minibatches.

Layer Norm (LN)

Aggregates across all channels C and spatial dimensions for each individual sample N. Independent of batch size, making it the standard for RNNs and Transformers.

Instance Norm (IN)

Aggregates strictly across spatial dimensions (H, W) for each single channel of each single sample. Widely used in generative style transfer.

Group Norm (GN)

Divides the channel dimension C into distinct groups and aggregates across each group per sample. Provides batch-size independence while preserving channel specialization.

—

1.9 Chapter Summary and Conceptual Graph

The foundations developed in this chapter form an interconnected chain where each new concept resolves a specific mathematical failure mode or constraint introduced by earlier building blocks:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** A single linear layer cannot approximate complex non-linear functions; we must stack layers into deep networks, which in turn requires Backpropagation to compute parameter gradients.
- **From Node 2 to 3:** Stacking linear transformations collapses into a single linear map ($W_2W_1x = W'x$); activation functions are strictly required to introduce non-linear expressivity.
- **From Node 3 to 4:** Deep non-linear networks are prone to signal decay at initialization. Naive random noise collapses activations to zero, causing vanishing gradients. Variance scaling (He/Xavier) dynamically stabilizes signal spread.
- **From Node 4 to 5:** With stable gradients established, evaluating the loss over millions of examples is computationally prohibitive; Stochastic Gradient Descent (SGD) enables rapid iterative updates via minibatches.
- **From Node 5 to 6:** High-capacity deep models easily overfit training data. Regularization (weight decay and Inverted Dropout) controls capacity by penalizing weights and enforcing distributed representations.
- **From Node 6 to 7:** Throughout deep optimization, intermediate layer distributions continuously drift; Batch, Layer, and Group Normalization re-center activations to maintain high learning rates and stable convergence.

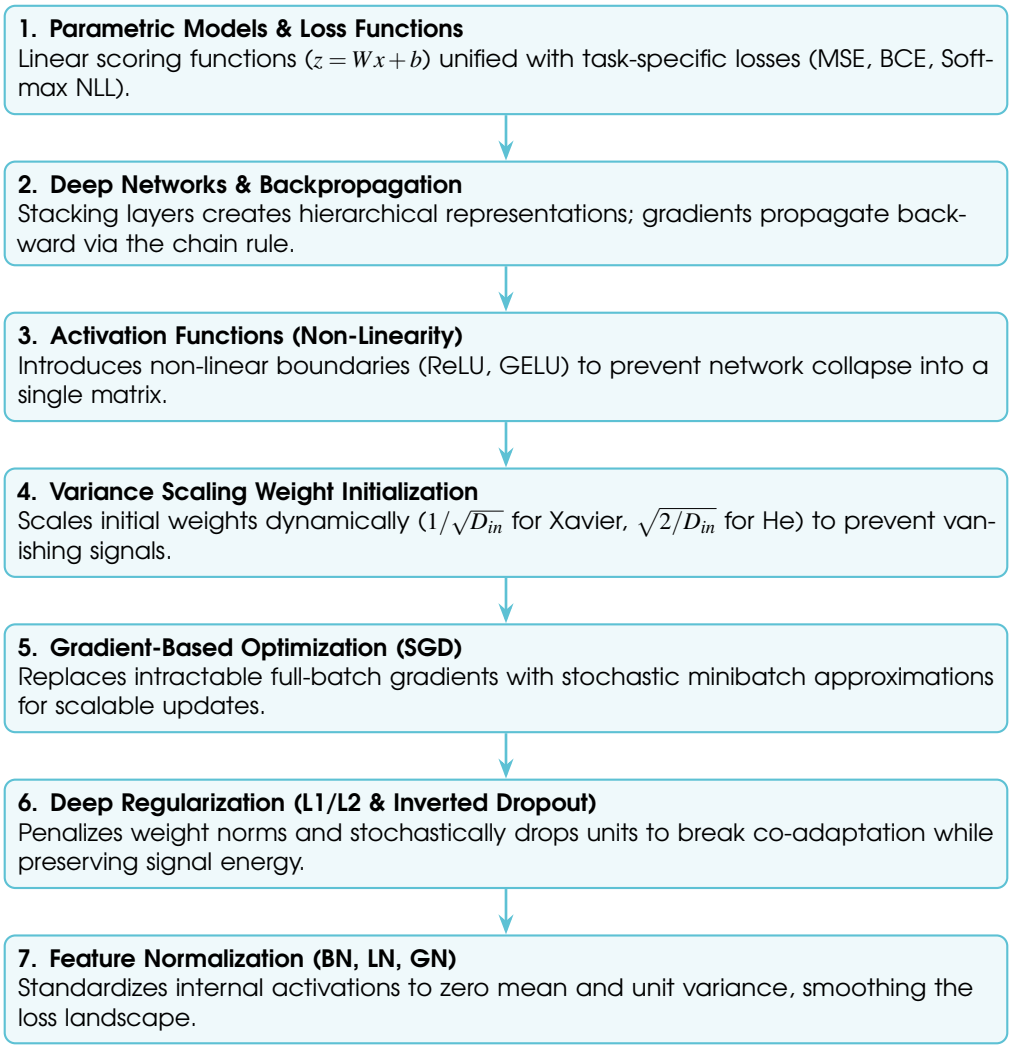
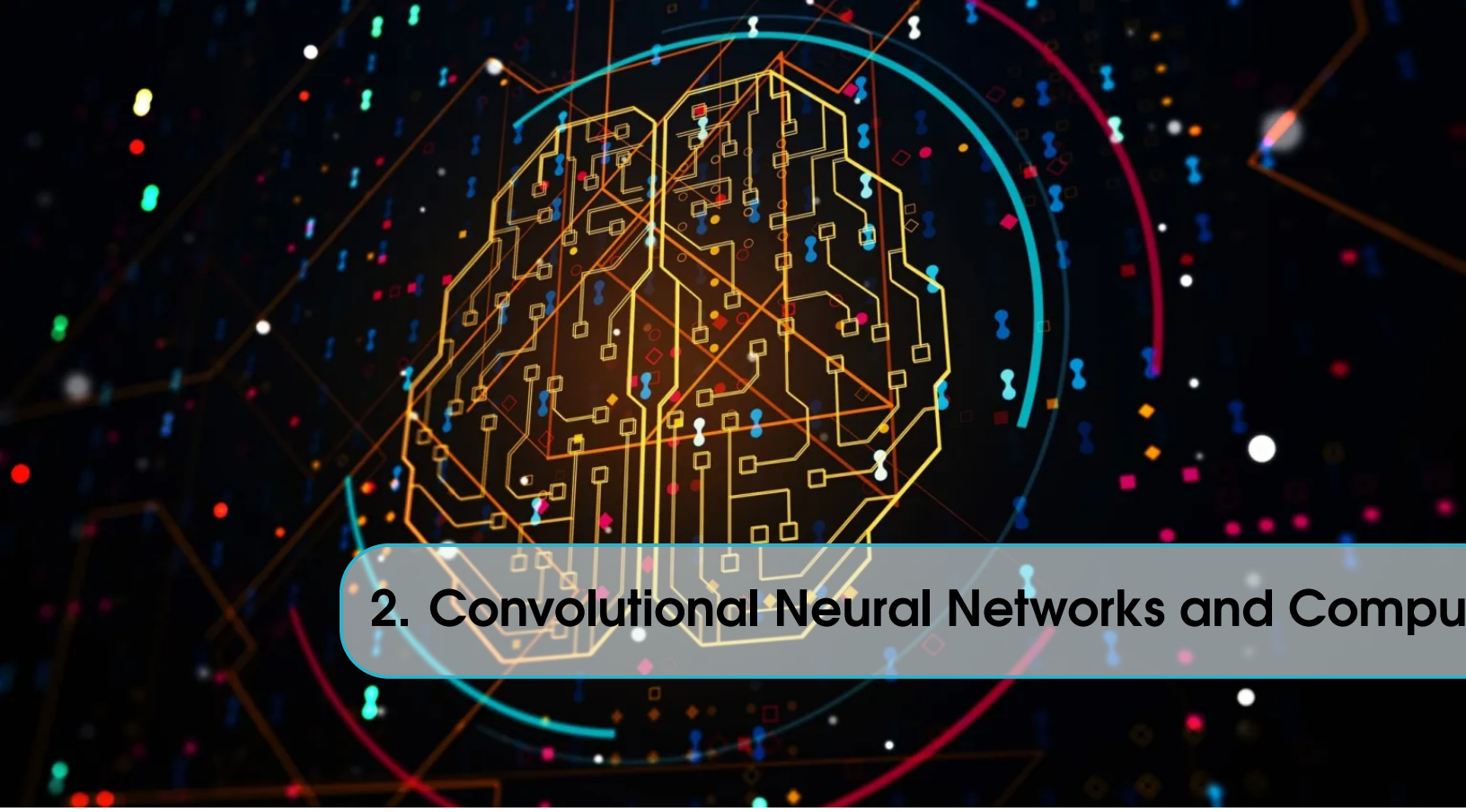


Figure 1.9: Conceptual roadmap of deep learning foundations and optimization.



2. Convolutional Neural Networks and Compu

2.1 Image Representation and the Imaging Process

In computer vision, the fundamental objective is to interpret and reason about visual inputs. Digital images are typically formed via an optical camera model (such as the pinhole camera model) and subsequently digitized, as shown in Figure 2.1.

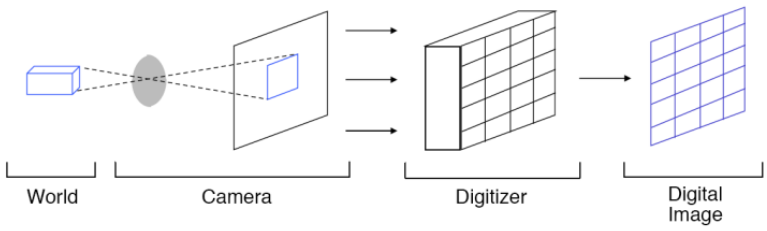


Figure 2.1: Illustration of the pinhole camera imaging process.

Note

The Digital Image Tensor: A digital camera digitizes incoming light into a 3D array of numbers with dimensions $(H \times W \times C)$, where H is the image height, W is the image width, and C represents the color channels (typically $C = 3$ for Red, Green, and Blue). Each pixel intensity is represented by an integer in $[0, 255]$ or normalized as a floating-point value in $[0, 1]$.

—

2.2 Motivations for Convolutions

2.2.1 The Problem with Fully Connected Layers

In standard Fully Connected (Dense) layers, every input unit is wired to every output unit. When applied directly to image data, this architecture exhibits severe failure modes:

- **Parameter Explosion & Overfitting:** Consider a modest 1000×1000 RGB image (3×10^6 input values). Connecting this input to a hidden layer of only 10^3 neurons requires over 3×10^9 parameters (3 billion weights). Storing and optimizing this volume of parameters requires unfeasible memory and inevitably triggers catastrophic overfitting.
- **Destruction of Spatial Topology:** Flattening an image into a 1D vector completely ignores spatial relationships, treating adjacent pixels with the same prior relevance as pixels located at opposite ends of the image.

2.2.2 Exploiting Image Structure: Locality and Stationarity

To overcome these structural limitations, **Convolutional Neural Networks (CNNs)** leverage two intrinsic physical properties of visual data:

1. **Local Spatial Correlation:** Spatially proximal pixels exhibit strong statistical correlation, whereas distant pixels are relatively uncorrelated. Instead of connecting each neuron globally to the entire image, a neuron connects only to a small local receptive field.
2. **Translation Stationarity (Invariance):** Visual features (such as edges, corners, and textures) are statistically invariant to spatial translation. A pattern detector useful in the top-left corner is equally meaningful in the bottom-right corner. Consequently, the same set of parameters can be shared across all spatial positions.

—

2.3 The Core Building Block: The Convolutional Layer

A Convolutional Layer replaces dense matrix multiplication with a local sliding operation, illustrated in Figure 2.2:

- **The Kernel (Filter):** A small matrix of learnable weights (e.g., 3×3 or 5×5). It serves as a specialized feature detector.
- **The Sliding Window:** The filter slides systematically across the spatial grid of the input. At each position, it computes the dot product (sum of element-wise multiplications) between its weights and the underlying local patch.

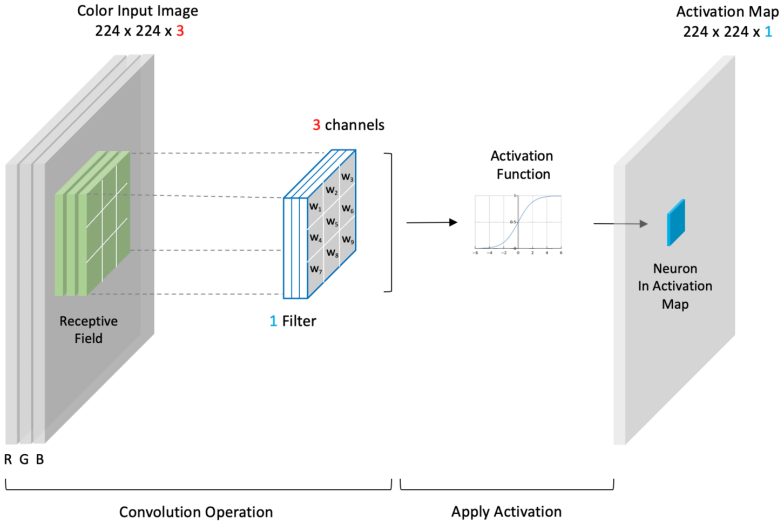


Figure 2.2: Illustration of a single convolutional layer operating on an image.

- **The Feature Map:** The resulting output grid, illustrated in Figure 2.3. High values indicate strong alignment between the learned filter pattern and that specific image region.

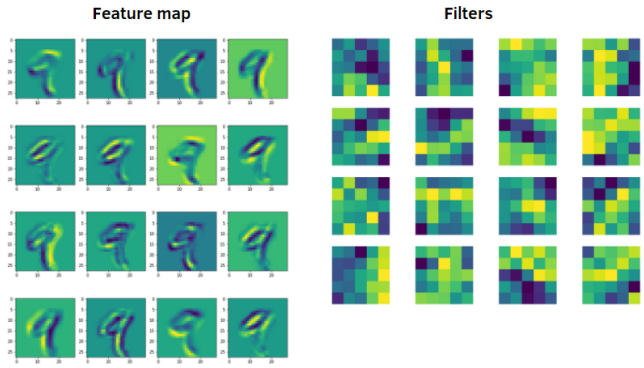


Figure 2.3: Multiple feature maps produced by convolving distinct filters over an input.

2.3.1 Key Hyperparameters of the Convolution

The behavior and spatial dimensions of a convolutional layer are governed by three primary hyperparameters:

- **Stride (S):** The step size with which the kernel moves across the image. A stride of $S = 1$ moves pixel-by-pixel, while $S > 1$ downsamples the output grid.
- **Padding (P):** Adding zero-valued borders around the input image boundary to preserve spatial dimensions and ensure edge pixels receive equal computational attention.
- **Channels (C):** Real images have depth (e.g., $C_{in} = 3$ for RGB). A kernel must match the depth of the input, yielding a 3D tensor of shape $(C_{in} \times K_h \times K_w)$. Using C_{out} distinct kernels produces C_{out} output feature

maps.

2.3.2 Formal Mathematical Definition

Formally, the discrete 2D convolution of an image I with a 2D filter g at spatial location $[m,n]$ is defined as:

$$f[m,n] = (I * g)[m,n] = \sum_k \sum_l I[m-k, n-l]\, g[k,l] \tag{2.1}$$

where k and l index the spatial extent of the kernel.

Key Word

Convolution versus Cross-Correlation in Deep Learning

In classical signal processing, the index subtraction $I[m-k, n-l]$ implies flipping the kernel g by 180°. Deep learning libraries (e.g., PyTorch) compute **Cross-Correlation** instead:

$$f[m,n] = (I \star g)[m,n] = \sum_k \sum_l I[m+k, n+l]\, g[k,l]$$

Because kernel weights $g[k,l]$ are learned from scratch via backpropagation, the network simply learns pre-flipped weights directly, rendering cross-correlation and convolution mathematically equivalent for optimization.

—

2.4 Classic Image Processing Filters

Before inspecting multi-channel deep representations, consider how classical handcrafted kernels manipulate local spatial neighborhoods to extract specific visual features.

Example

1. Smoothing Filter (Noise Reduction):

Replacing each pixel with a weighted neighborhood average dampens noise and sharp outliers:

2	3	3
3	**20**	2
3	2	3

Smoothing →

2	3	3
3	**3**	2
3	2	3

The central anomaly (20) is effectively smoothed down to 3.

Example

2. Vertical Edge Detection Filter:

Filters with opposing positive and negative signs detect sharp directional variations in pixel intensity:

$$g_{\text{vertical}} = \begin{array}{|c|c|c|}\hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array}$$

Transposing g_{vertical} yields a horizontal edge detector $g_{\text{horizontal}} = g_{\text{vertical}}^T$.

Example

3. Step-by-Step 2D Convolution Example (Stride 1, Pad 1):

Convolving a 3×3 image I with a vertical edge filter g with padding $P = 1$ pads the input with zeros into a 5×5 grid I_{padded}:

$$I_{\text{padded}} = \begin{array}{|c|c|c|c|c|}\hline 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 8 & 5 & 2 & 0 \\ \hline 0 & 7 & 5 & 3 & 0 \\ \hline 0 & 9 & 4 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 \\ \hline \end{array}$$

Applying the convolution with stride $S = 1$ yields:

$$\underset{g[k,l]}{\begin{array}{|c|c|c|}\hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array}} \otimes \underset{\text{(conceptually)}}{I_{\text{padded}}} = \underset{f[m,n]}{\begin{array}{|c|c|c|}\hline -10 & 10 & 10 \\ \hline -14 & 18 & 14 \\ \hline -9 & 12 & 9 \\ \hline \end{array}}$$

Accounting for the 180° Kernel Flip:

Flipping g by 180° produces:

$$g_{\text{flipped}} = \begin{array}{|c|c|c|}\hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array}$$

- **Center Value $f[1,1] = 18$:** Flipped kernel overlaps fully with the original 3×3 image pixels:

$$(1 \cdot 8 - 1 \cdot 2) + (1 \cdot 7 - 1 \cdot 3) + (1 \cdot 9 - 1 \cdot 1) = 6 + 4 + 8 = 18$$

- **Top-Left Value $f[0,0] = -10$:** Flipped kernel overlaps with the top-left boundary of I_{padded}:

$$(1 \cdot 0 - 1 \cdot 5) + (1 \cdot 0 - 1 \cdot 5) = -10$$

—

2.5 Output Shape and Parameter Counting for 3D Volumes

In deep vision models, inputs are 3D tensors $(C_{\text{in}} \times H \times W)$, and each layer applies a bank of C_{out} distinct filters.

2.5.1 Spatial Output Size Formula

Given an input spatial width W, kernel spatial size K, padding P, and stride S, the output spatial width W_{out} is given by:

$$W_{\text{out}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \tag{2.2}$$

Example

Numerical Example of Parameter Counting:
Suppose the input volume is a $32 \times 32 \times 3$ image, and we apply 10 filters of size 5×5 (with stride $S = 1$, padding $P = 2$):

- **Output spatial size:** $\lfloor (32 - 5 + 2 \cdot 2)/1 \rfloor + 1 = 32 \implies (32 \times 32)$.
- **Output volume size:** $32 \times 32 \times 10$.
- **Number of parameters:** Each filter has $5 \times 5 \times 3 = 75$ weights plus 1 bias $= 76$ parameters. For 10 filters: $76 \times 10 = \mathbf{760}$ parameters.

—

2.6 The Pooling Layer

Convolutional neural networks interleave convolutional layers with **Pooling layers** to perform spatial down-sampling.

2.6.1 Purpose and Properties of Pooling

- **Spatial Dimensionality Reduction:** Reduces spatial width and height, reducing computational complexity in deeper layers.
- **Parameter Control:** Shrinking feature map resolutions minimizes the number of parameters needed when eventually transitioning to fully connected classification heads.
- **Translation Invariance:** Pooling provides slight invariance to local translations; small shifts in the input do not alter the pooled maximum or mean.
- **Channel Independence:** Pooling operates independently on each depth slice without mixing channels. If the input has C channels, the output has exactly C channels.

2.6.2 Max Pooling

Max Pooling slides a window across each channel and outputs strictly the **maximum value** in each receptive window.

Example

Max Pooling Example (2×2 filter, Stride 2):
Applying Max Pooling to a 4×4 slice:

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

$\xrightarrow{\text{Max Pool } 2\times2, S=2}$

6	8
3	4

Spatial dimensions are halved, discarding 75% of activations and retaining only the most prominent local features.

2.6.3 Average Pooling

Average Pooling computes the **arithmetic mean** over each local window:

Example

Average Pooling Example (2×2 filter, Stride 2):
Applying Average Pooling to a 4×4 slice:

1	1	2	4
4	6	6	8
3	2	1	0
1	2	3	4

$\xrightarrow{\text{Average Pool } 2\times2, S=2}$

3	5
2	2

—

2.7 Visualizing Convolutional Networks

Key Insight

Visualizing what deep CNNs learn (e.g., via Deconvolutional Networks by Zeiler and Fergus) reveals a clear hierarchical decomposition:

- **Layer 1:** Learns elementary low-level visual primitives (oriented edges, basic color gradients).
- **Layer 2:** Combines primitives into textures, corners, and simple geometric patterns.
- **Layer 3 and Deeper:** Assembles textures into semantically complex object parts (e.g., wheels, faces) and eventually full objects.

—

2.8 Vision Regularization: Data Augmentation

While weight decay acts on model parameters and Dropout acts on activations, **Data Augmentation** acts directly on the input data. It artificially expands training dataset diversity by applying label-preserving

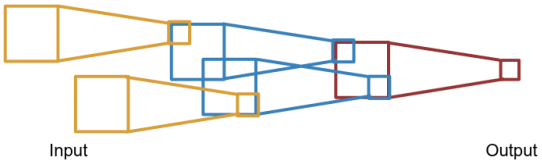


Figure 2.4: Hierarchical feature representations learned across deep convolutional layers.

transformations, embodying the standard modern regularization paradigm: introducing stochasticity at training and evaluating on fixed standard inputs at test time.

2.8.1 Classic Spatial and Color Augmentations

- **Horizontal Flips:** A simple, highly effective transformation for general object recognition.
- **Random Crops and Scales:** During training, random patches are extracted at varying scales (e.g., ResNet resizes the short side between 256 and 480 pixels before extracting a 224×224 crop). At test time, predictions are averaged over a fixed set of crops (e.g., 10 crops: 4 corners + center, and their horizontal flips).
- **Color Jitter:** Perturbing brightness, contrast, and saturation to render the model invariant to illumination changes.

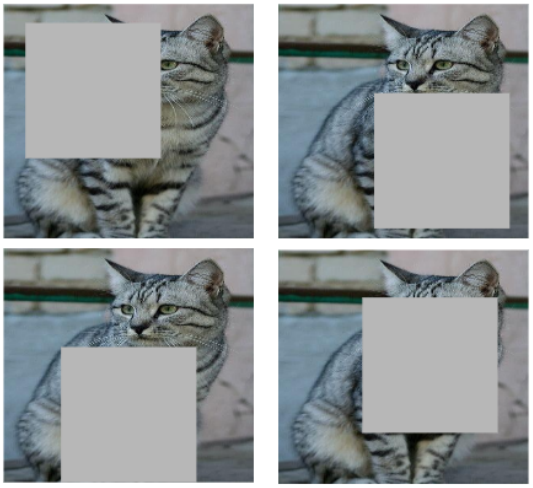


Figure 2.5: Example of Cutout.

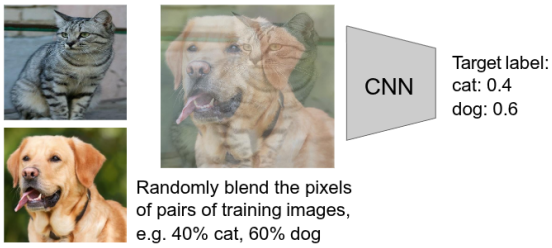


Figure 2.6: Example of Mixup.

2.8.2 Advanced Stochastic Augmentations

- **Cutout:** During training, square image patches are randomly masked out (set to zero). This forces the network to avoid over-relying on a single dominant feature (such as an animal's face) and learn distributed contextual cues. Unmodified images are evaluated at test time.
- **Mixup (Zhang et al., 2018):** Blends pairs of images and their corresponding one-hot labels linearly. For instance, blending 40% of a cat image with 60% of a dog image produces a target training label of $[0.4\,\text{cat}, 0.6\,\text{dog}]$. At test time, standard unblended images are used.

—

2.9 Transfer Learning in Computer Vision

2.9.1 The Fundamental Concept (Why Use It?)

Training deep Convolutional Neural Networks from scratch requires massive datasets (e.g., millions of annotated images). On small target datasets, training from scratch leads to severe overfitting.

The solution is **Transfer Learning**: leveraging a **Source model** pre-trained on a gigantic dataset (e.g., ImageNet) and transferring its learned visual representations to a **Target model** for a specific, smaller task.

2.9.2 Practical Implementation

Using standard backbones like VGG or ResNet, transfer learning proceeds in two stages:

- **1. Preserving Convolutional Feature Extractors:** Early and middle convolutional layers are kept frozen. They act as universal feature extractors, having already learned edge, texture, and shape primitives common to all natural visual scenes.
- **2. Replacing the Classification Head:** The final Fully Connected layers are removed and replaced with new layers configured for the target dataset's classes. Only this new head (and optionally fine-tuned upper conv layers) is trained.

2.9.3 Pervasiveness: The Modern Standard

Transfer learning is standard across all computer vision applications:

- **Object Detection:** Frameworks like Fast R-CNN utilize ImageNet-pre-trained CNN backbones to extract spatial features before predicting bounding boxes.
- **Image Captioning:** Pre-trained CNN backbones produce dense feature vectors that initialize or condition recurrent sequence decoders.

Key Word

The Model Zoo

All major deep learning frameworks provide a **Model Zoo** (e.g., `torchvision.models` in PyTorch) containing pre-trained weights for state-of-the-art vision architectures. These models can be downloaded instantly and fine-tuned for custom downstream tasks.

2.10 Chapter Summary and Conceptual Graph

The concepts developed in this chapter establish how the inductive biases of natural visual data lead from raw pixels to deep hierarchical vision systems:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** Viewing digital images as raw flattened arrays reveals why generic architectures fail: high spatial resolutions lead directly to parameter explosion and loss of geometry.
- **From Node 2 to 3:** Recognizing the failure of dense connections motivates identifying domain-specific inductive biases: pixels exhibit local correlation and patterns are invariant to spatial shifts.
- **From Node 3 to 4:** Locality and stationarity directly dictate the mathematical structure of the Convolutional Layer: local receptive fields with shared kernel weights.
- **From Node 4 to 5:** While convolutions preserve spatial structure, deep networks need downsampling to grow the effective receptive field and limit computation, achieved through parameter-free Pooling layers.
- **From Node 5 to 6:** Interleaving convolutions and pooling enables deep networks to naturally learn hierarchical representations, moving from oriented edges in early layers to semantic concepts in deep layers.
- **From Node 6 to 7:** Deep vision models with high expressive capacity easily memorize training images; Data Augmentation (including Cutout and Mixup) enforces invariance to photometric and spatial perturbations.
- **From Node 7 to 8:** Because training deep CNNs from scratch requires millions of images, Transfer Learning allows small custom datasets to inherit universal visual features from large-scale pre-trained Model Zoos.

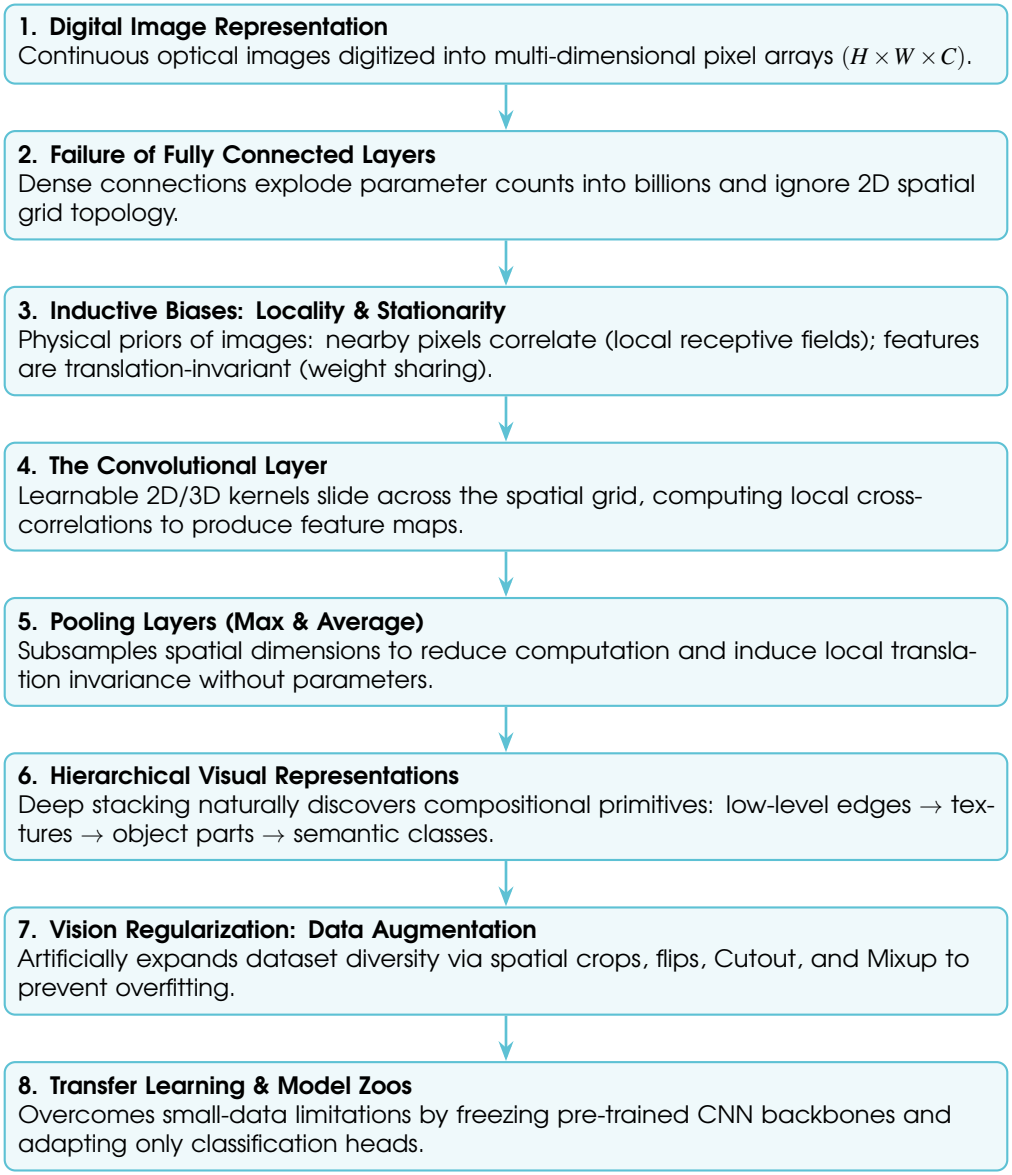


Figure 2.7: Conceptual roadmap of convolutional neural networks and computer vision.



3. Sequence Models

3.1 Introduction to Sequence Modeling

Standard feedforward architectures (such as multi-layer perceptrons and traditional convolutional neural networks) operate strictly under a **one-to-one** paradigm: a fixed-dimensional input vector $x \in \mathbb{R}^D$ is mapped to a fixed-dimensional output $y \in \mathbb{R}^K$. While effective for static tasks like single-image classification, this formulation fails when dealing with data that unfolds over time or possesses an intrinsic sequential nature, such as text, audio, video streams, or financial time series.

Key Word

The Sequence Modeling Challenge

In sequential tasks, either the input, the output, or both can have arbitrary and variable lengths:

$$\{x_1, x_2, \ldots, x_T\} \quad \longrightarrow \quad \{y_1, y_2, \ldots, y_{T'}\}$$

A sequence model must be able to preserve temporal order, handle variable-length sequences without retraining, and share learned parameters across different time steps.

3.1.1 Taxonomy of Sequence Tasks

Depending on whether the input and output consist of single vectors or sequences of vectors, sequential problems are traditionally classified into five primary architectural paradigms, illustrated in Figure 3.1.

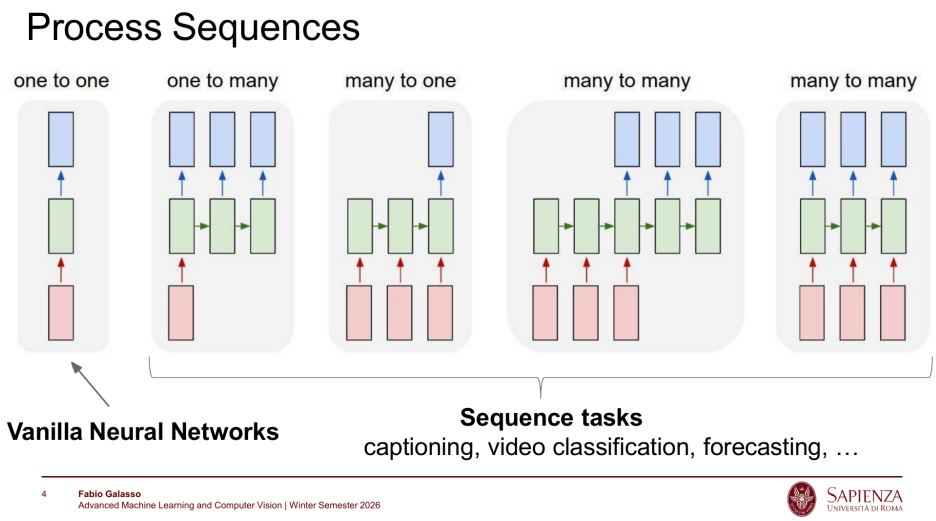


Figure 3.1: Architectural taxonomy of sequence processing tasks (from course slides). Pink boxes denote inputs, green boxes represent internal processing units, and blue boxes denote outputs.

- **One-to-One:** The conventional neural network setup without temporal recurrence. A single input produces a single output (e.g., standard image classification).
- **One-to-Many:** A single fixed-size input generates a sequential output of variable length. A prominent example is **Image Captioning**, where a single visual input (e.g., a photo of a dog) produces an output sequence of textual tokens (*"A happy brown dog"*).
- **Many-to-One:** A temporal sequence of inputs is processed and summarized into a single static prediction. Typical examples include **Action Prediction / Video Classification** (where a temporal sequence of consecutive video frames is mapped to a single discrete action label like *"Jumping"*) and **Sentiment Analysis** (a sequence of words mapped to a positive or negative score).
- **Many-to-Many (Synchronized / Aligned):** Both input and output are sequences of equal length, evaluated in lockstep at every time step. An example is **Frame-level Video Classification**, where an action category or segmentation map is predicted synchronously for each incoming frame.
- **Many-to-Many (Asynchronous / Sequence-to-Sequence):** The input sequence and output sequence have independent lengths ($T \neq T'$). This paradigm powers **Machine Translation** (translating an English sentence into Italian) and **Video Captioning** (ingesting a sequence of video frames to output a descriptive sentence, e.g., *"A dog jumps over a hurdle"*).

3.1.2 Forecasting as a Core Sequence Problem

A critical subset of sequence tasks is **Forecasting**: given a historical trajectory or sequence of observations up to the current time t, predict future observations:

$$(x_1, x_2, \ldots, x_t) \quad \longrightarrow \quad (\hat{x}_{t+1}, \hat{x}_{t+2}, \ldots)$$

Example

Real-World Applications of Forecasting:

- **Trajectory Forecasting:** Predicting the future spatial paths of pedestrians and vehicles, essential for collision avoidance in autonomous driving and human-robot interaction.
- **Long-term Tracking across Occlusions:** Estimating the motion of targets when temporarily blocked by obstacles.
- **Physical and Financial Time Series:** Forecasting weather patterns, seismic alerts, critical pressure thresholds in industrial power plants, and financial market trends.

—

3.2 Temporal Convolutional Networks

Before exploring recurrent architectures, sequence modeling can be approached using **Convolutional Neural Networks** applied along the temporal dimension, commonly known as **Temporal Convolutional Networks (TCN)**.

3.2.1 One-Dimensional Convolutions on Sequences

While standard computer vision uses 2D convolutions over spatial grids, temporal data is structured as a 1D sequence over discrete time. The 1D discrete convolution of an input sequence f with a learnable filter g is formulated as:

$$(f * g)(i) = \sum_{k} g(k)\, f(i-k) \tag{3.1}$$

Note

Piece-by-Piece Mathematical Breakdown of Equation 3.1:

- i: The discrete temporal index (time step) at which the output feature is evaluated.
- f: The 1D input sequence signal across time (e.g., raw temporal measurements or feature embeddings).
- g: The learnable convolutional kernel (filter) of fixed size K.
- k: The kernel offset index running over the temporal window of the filter ($k \in \{0, \ldots, K-1\}$).
- $g(k)$: The learnable parameter (weight) at relative offset k within the kernel.
- $f(i-k)$: The input value sampled k time steps prior to time step i.
- $\sum_k$: The summation accumulating the weighted products across the entire receptive span of the kernel.

3.2.2 Causal and Dilated Convolutions

When applying convolutions to sequential forecasting, two structural constraints arise:

Causal Convolutions

Standard convolutions compute outputs using a symmetric window that looks both backward and forward in time. For temporal prediction, accessing future time steps ($i+k$ with $k>0$) causes an illegal information leak (*lookahead*). Therefore, the convolution must be strictly **causal**: the calculation at time step t depends solely on inputs x_s where $s \leq t$.

Dilated Convolutions and Receptive Field

A fundamental limitation of standard causal convolutions is that the **receptive field** (the maximum historical context the network can observe) grows only *linearly* with the depth of the network:

$$\text{Receptive Field} = 1 + L(K-1)$$

where L is the number of layers and K is the kernel size. Capturing long-term temporal dependencies would therefore require an impractical number of layers.

To overcome this limitation, TCNs introduce **Dilated Causal Convolutions**, as illustrated in Figure 3.2. For a 1D sequence f and kernel g, a dilated convolution with dilation factor d is defined as:

$$(f *_d g)(i) = \sum_k g(k)\, f(i - d \cdot k) \tag{3.2}$$

Note

Piece-by-Piece Mathematical Breakdown of Equation 3.2:

- d: The **dilation factor**, which determines the spacing between kernel taps. For $d=1$, Equation 3.2 reduces to a standard convolution. For $d>1$, the filter samples inputs every d steps, skipping $d-1$ intermediate values.
- $i - d \cdot k$: The temporal sampling index, stretching the effective reach of the kernel backwards in time without adding extra parameters.

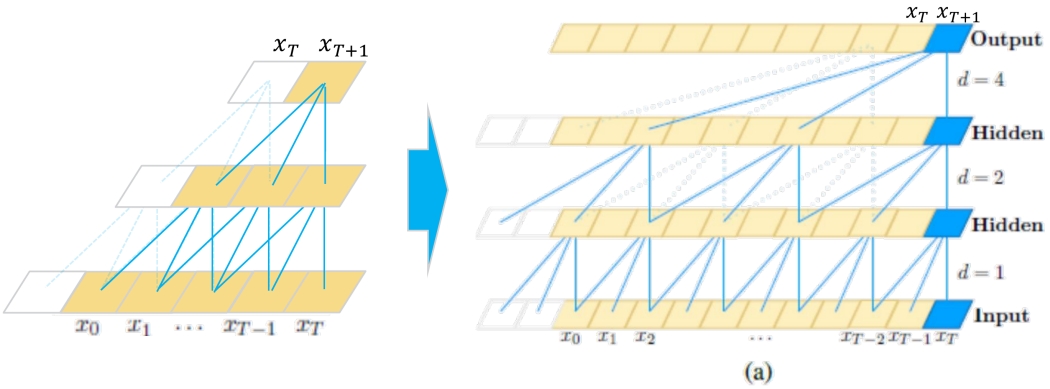


Figure 3.2: Architectural comparison between standard causal convolutions and dilated causal convolutions (Bai et al., 2018). Left: Standard causal convolution where receptive field grows linearly with depth. Right: Dilated causal convolution with exponentially doubling dilation factors ($d = 1, 2, 4$), achieving exponential receptive field expansion without gaps in the input history.

Comparative Architectural Analysis: Standard vs. Dilated Causal Convolutions

Figure 3.2 directly visualizes the paradigm shift from standard causal convolutions (left) to the dilated architecture proposed by Bai et al. (right):

- **Left Panel — Standard Causal Convolution (Linear Interaction Distance):**
  - *Strict Causality:* Time flows from left to right ($x_0 \to x_1 \cdots \to x_T$). Predictions evaluated at time t must never access future observations (x_{t+1}, x_{t+2}). Consequently, the convolutional kernel is strictly causal: connections from upper layers descend exclusively vertically or diagonally towards the left (past), ensuring zero lookahead.
  - *The Receptive Field Bottleneck:* With a standard kernel size K and no dilation ($d = 1$), each additional layer increases the receptive field by only $K - 1$ past time steps:

$$\text{Receptive Field} = 1 + L(K - 1)$$

    As visible in the narrow triangular pyramid on the left, capturing a historical horizon of N time steps requires $L \approx N/(K - 1)$ layers. For long temporal sequences (e.g., $N = 1000$), an impractically deep network of hundreds of layers would be required, introducing severe computational costs and optimization difficulties.
- **Right Panel — Dilated Causal Convolution (Exponential Receptive Field):** Instead of growing the filter size or stacking hundreds of layers, the network keeps small, efficient kernels (e.g., $K = 2$ parameters per channel) while exponentially doubling the dilation factor d at each successive layer:
  1. **Input Layer (bottom):** Represents the raw discrete input sequence $(x_0, x_1, \ldots, x_T)$.
  2. **First Hidden Layer ($d = 1$):** Standard stride-1 causal convolution. Each node aggregates 2 consecutive values (x_t and x_{t-1}).
  3. **Second Hidden Layer ($d = 2$):** The dilation factor doubles to $d = 2$. The filter spans 2 positions spaced by 2 time steps (sampling t and $t - 2$, skipping the intermediate point).
  4. **Output Layer ($d = 4$):** The dilation factor doubles again to $d = 4$. The filter samples features spaced 4 time steps apart (at t and $t - 4$).

- **Tracing the Blue Receptive Tree (Top-Right Node at x_T):** Following the blue highlighted connections backwards in Figure 3.2 illustrates how exponential dilation covers the past:
  - At layer $d = 4$, the top-right output node branches into **2 features** (at T and $T-4$).
  - At layer $d = 2$, each of these two features branches into 2 nodes at stride 2, yielding **4 features** $(T, T-2, T-4, T-6)$.
  - At layer $d = 1$, each of these four features branches into 2 adjacent nodes, covering **8 consecutive input tokens** $(x_{T-7}, x_{T-6}, \ldots, x_T)$.

  With only 3 convolutional layers and $K = 2$, the receptive field spans 8 full time steps.

Key Word

Why Dilation is Logarithmic and Continuous (w/o Gaps)

1. Logarithmic Depth Requirement: Because the effective receptive span doubles with every layer $(1, 2, 4, 8, \ldots)$, the receptive field scales exponentially with depth:

$$\text{Receptive Field} = 1 + (K-1)\sum_{l=0}^{L-1} 2^l = 1 + (K-1)(2^L - 1)$$

To observe a temporal context of length N, the required network depth scales **logarithmically**:

$$L \approx \log_2(N)$$

For instance, while a standard causal convolution would require ≈ 1000 layers to observe 1000 historical steps, a dilated TCN achieves this with just $\lceil \log_2(1000) \rceil = 10$ layers!

2. Coverage Without Gaps (*w/o gaps*): Although individual higher-level filters skip intermediate activations, the hierarchical doubling of dilation factors ($d_l = 2^l$) ensures that every single discrete input token within the receptive span contributes to the final prediction. There are no temporal holes or unobserved steps in the input history.

3.2.3 Trade-offs of Temporal Convolutional Networks

Key Insight

Advantages and Disadvantages of TCNs:

- **Pros:**
  - **Parallelization:** Unlike recurrent architectures, convolutions can be computed concurrently across all time steps during training, drastically accelerating training times.
  - **Local Dependencies:** Convolutions excel at modeling localized temporal features and patterns.
- **Cons:**
  - **Fixed Maximum Memory:** The receptive field is strictly bounded by the network depth and dilation schedule; historical information beyond this window is permanently lost.
  - **Inflexibility at Test Time:** Handling variable sequence lengths or streaming evaluation requires managing historical buffers, whereas recurrent models update a single internal state vector dynamically.

3.3 Recurrent Neural Networks

The primary and historically dominant paradigm for processing sequential data is the **Recurrent Neural Network (RNN)**. Instead of relying on expanding spatial filters, an RNN introduces an internal **hidden state** that acts as a dynamic memory buffer.

3.3.1 The Recurrence Principle

At each time step t, the network processes an incoming input x_t while retaining a memory vector of past computations, denoted as the **hidden state** h_{t-1}. The state is updated via a recurrence formula:

$$h_t = f_W(h_{t-1}, x_t) \tag{3.3}$$

When output predictions are required, the current hidden state is mapped to an output vector:

$$y_t = f_{W_{hy}}(h_t) \tag{3.4}$$

Note

Piece-by-Piece Breakdown of Equations 3.3 and 3.4:

- t: The current discrete time step index ($t = 1, 2, \ldots, T$).
- h_t: The **new hidden state** vector at time t, capturing the historical context up to the current moment.
- h_{t-1}: The **previous hidden state** vector from time step $t-1$ (with h_0 typically initialized to zero).
- x_t: The **input vector** fed into the network at time step t.
- f_W: A parameterized non-linear state transition function. **Crucial property:** *the exact same function f_W and the exact same set of parameters W are reused identically across every time step.*
- y_t: The predicted output vector (e.g., class logits) generated at time step t.
- $f_{W_{hy}}$: The output mapping function parameterized by weights W_{hy}.

3.3.2 The Vanilla RNN Formulation

In the classic formulation (often called the **Vanilla RNN** or **Elman RNN**), the transition function applies affine transformations followed by a hyperbolic tangent (tanh) non-linearity:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t) \tag{3.5}$$

$$y_t = W_{hy} h_t \tag{3.6}$$

Note

Piece-by-Piece Mathematical Breakdown of Equation 3.5:

- W_{hh}: The **hidden-to-hidden weight matrix**. It transforms the previous memory vector h_{t-1}, governing how historical context propagates into the future.
- W_{xh}: The **input-to-hidden weight matrix**. It projects the incoming input vector x_t into the hidden feature space.
- tanh: The hyperbolic tangent activation function. It squashes state values strictly into the interval

$(-1, 1)$, preventing activations from growing unbounded across successive steps.

- W_{hy}: The **hidden-to-output weight matrix**. It projects the internal hidden representation h_t into the target output space.

3.3.3 Unrolling the Computational Graph

To understand how an RNN is trained, the recurrence is unrolled across time into a directed acyclic computational graph, as shown in Figure 3.3.

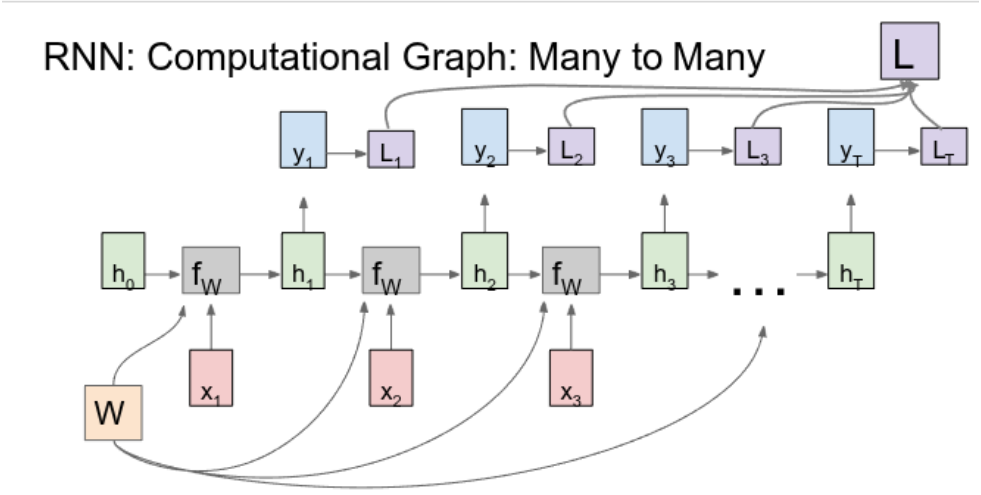


Figure 3.3: Unrolled computational graph of a recurrent neural network across time steps $t = 1, \ldots, T$. Notice that the weight matrix W is reused identically at every time step.

At each time step t, the network receives input x_t, updates its hidden state h_t, and optionally emits an output y_t. If a target label y_t^* is available, a local loss L_t is computed. The total loss across the full sequence of length T is the accumulated sum:

$$L = \sum_{t=1}^{T} L_t$$

3.3.4 Sequence-to-Sequence: Synchronized vs. Encoder-Decoder Mappings

When modeling sequences with inputs and outputs, two distinct structural paradigms emerge from the computational graph, as introduced in the lecture slides (Slides 41–46):

Synchronized Many-to-Many (Slide 41)

In a synchronized recurrence, the network ingests an input x_t and immediately emits a corresponding output y_t at every time step t. While effective for tasks with strict 1:1 temporal alignment (e.g., frame-level video classification or part-of-speech tagging), this architecture fundamentally fails for tasks such as **Machine Translation**:

- **Length Disparity ($T_{\text{in}} \neq T_{\text{out}}$):** Natural languages rarely have matching word counts (e.g., English *"I am hungry"* [3 words] $\longrightarrow$ Italian *"Ho fame"* [2 words]). Synchronous evaluation cannot handle length discrepancies.

- **Structural Word Reordering:** Grammar and syntax vary across languages. In English, adjectives precede nouns (*"the black cat"*), whereas in Italian they follow (*"il gatto nero"*). A synchronized network would be forced to predict the first translated word before even observing the subsequent subject.

Sequence-to-Sequence: Many-to-One + One-to-Many (Slides 45–46)

To resolve these limitations, Sutskever et al. (2014) proposed the **Sequence-to-Sequence (Seq2Seq)** framework, decomposing the task into two decoupled recurrent modules operating in cascade, illustrated in Figure 3.4:

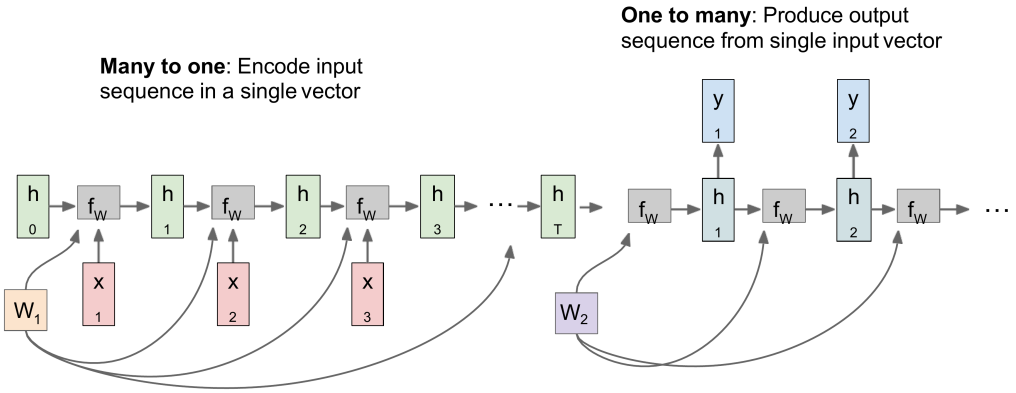


Figure 3.4: Sequence-to-Sequence architecture (Sutskever et al., 2014, from course slides). The Encoder (Many-to-One with weights W_1) compresses the source sequence into bottleneck state h_T. The Decoder (One-to-Many with weights W_2) generates the target sequence from h_T until an end-of-sequence token is emitted.

1. **Encoder — Many-to-One (Parameterized by W_1):**
   The encoder consumes the entire input sequence $x_1, \ldots, x_T$ step-by-step without producing intermediate outputs. Its sole purpose is to accumulate the full contextual meaning of the input into the final hidden state vector h_T, known as the **context** or **bottleneck vector**.
2. **Decoder — One-to-Many (Parameterized by W_2):**
   The decoder receives h_T as its initial hidden state and autoregressively generates the output sequence $y_1, y_2, \ldots, y_{T'}$. Generation terminates dynamically when the model emits an end-of-sequence token $\langle \text{END} \rangle$, allowing T' to differ arbitrarily from T.

Key Word

Why Two Distinct Weight Sets ($W_1 \neq W_2$)?

The encoder weights W_1 and decoder weights W_2 are explicitly decoupled:

- W_1 learns temporal dynamics, grammar, and semantic representations of the **source** domain (e.g., parsing English syntax).
- W_2 learns the vocabulary distributions, syntax rules, and generation dynamics of the **target** domain (e.g., generating fluent Italian sentences).

3.3.5 Case Study: Character-Level Language Modeling

To see an RNN in action, consider a character-level language model trained to predict the sequence *"hello"* from a vocabulary of four letters: $\{h, e, l, o\}$ (Slides 47–53). At each step, the network receives the current character and predicts a probability distribution over the vocabulary for the next character.

Training Phase: Teacher Forcing

During training, the complete sequence *"hello"* is known in advance. The model is trained using **Teacher Forcing** (also called *Ground Truth Forcing*), illustrated in Figure 3.5.

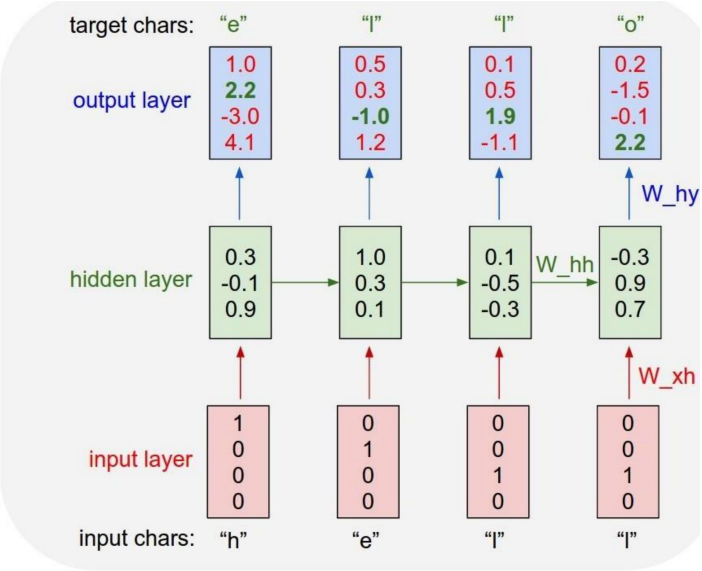


Figure 3.5: Training phase of the character-level RNN with Teacher Forcing (Slide 49). At each step, the ground-truth character from the dataset is fed as input, regardless of what the network predicted at the previous step.

As shown in Figure 3.5:

- **Network Flow:** Each character is represented as a one-hot vector in the input layer (4 red nodes), processed by the recurrent hidden layer (3 green nodes), and projected to raw scores (logits) in the output layer (4 blue nodes).
- **Teacher Forcing:** At each step t, the input is strictly the ground-truth character from the target word:
  - Feed *'h'* $\longrightarrow$ target is *'e'*
  - Feed *'e'* $\longrightarrow$ target is *'l'*
  - Feed *'l'* $\longrightarrow$ target is *'l'*
  - Feed *'l'* $\longrightarrow$ target is *'o'*

  Even if the network makes an incorrect prediction at step 1, step 2 is still fed the true character *'e'*. This prevents errors from accumulating during training.
- **Memory Disambiguates Consecutive Letters:** Notice steps 3 and 4: both receive the identical input vector for *'l'*, but step 3 must predict *'l'* while step 4 must predict *'o'*. A static feedforward network cannot tell them apart. The RNN succeeds because its hidden state acts as memory: step 3 remembers the past context $[h \to e \to l]$, whereas step 4 remembers $[h \to e \to l \to l]$.

Test Phase: Autoregressive Generation

At test time, the model must generate text autonomously without ground-truth answers. It operates in an **Autoregressive Loop**, shown in Figure 3.6.

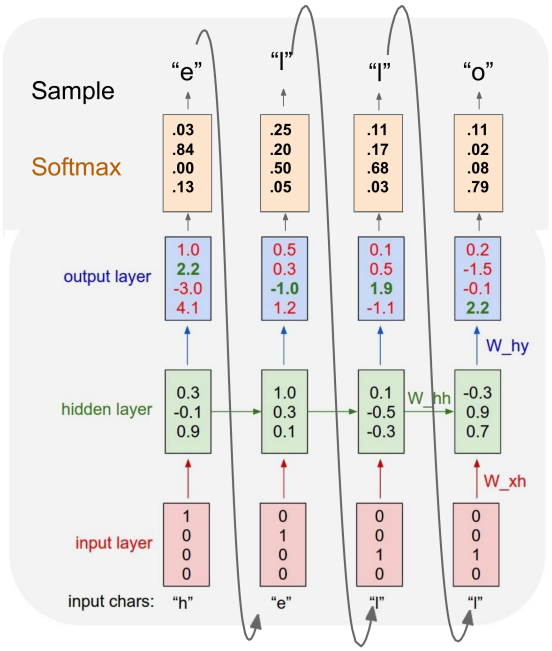


Figure 3.6: Test phase of the character-level RNN (Slide 53). Characters sampled from the output distribution are fed back recursively as inputs for the next time step (curved loop arrows).

The generation proceeds step by step:

- **Recursive Feedback:** Because there is no teacher at test time, the character produced at step t is fed back as the input for step $t+1$, as traced by the curved arrows in Figure 3.6.
- **Sampling from Softmax:**
  1. **Seed Prompt ($t=1$):** We provide an initial character, $x_1 =$ *'h'*. The network computes output scores and converts them to probabilities via Softmax (yielding $P(e) = 0.84$). We sample token *'e'*.
  2. **Step $t=2$:** The sampled *'e'* is fed back as input x_2. Combining it with the internal hidden state, the network predicts probabilities and samples *'l'* ($P(l) = 0.50$).
  3. **Step $t=3$:** The sampled *'l'* is fed back as input x_3, yielding another sampled *'l'* ($P(l) = 0.68$).
  4. **Step $t=4$:** The second *'l'* is fed back as input x_4. Thanks to the internal memory accumulated over time, the network recognizes the completed prefix and samples *'o'* ($P(o) = 0.79$).

Key Word

Training vs. Testing at a Glance

- **Training (Teacher Forcing):** Inputs come from the real dataset. Errors do not compound because the ground truth is provided at every step.
- **Testing (Autoregressive):** Inputs come from the model's own previous outputs. The network generates sequences recursively on its own.

3.3.6 Case Study: Image Captioning (CNN + RNN)

Another foundational application of recurrent models is **Image Captioning** (Slides 112–123): given an input image, the network must generate a fluent sentence describing its visual content. This represents a canonical **One-to-Many** task that unifies Computer Vision (CNN) and Natural Language Processing (RNN).

How CNN and RNN Interact

The architecture combines a Convolutional Neural Network (as a visual feature extractor) and a Recurrent Neural Network (as a sequential language generator), as shown in Figure 3.7.

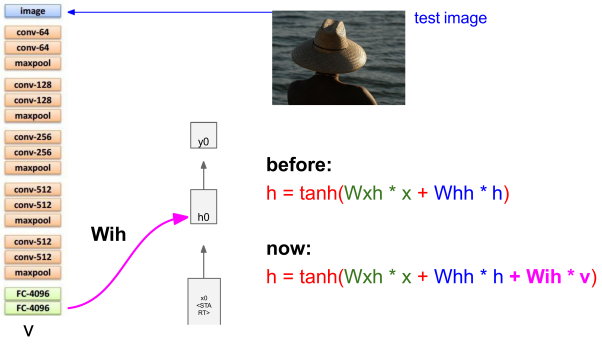


Figure 3.7: Interaction between CNN and RNN for Image Captioning (Slide 118). The CNN extracts visual feature vector v from a pre-trained layer, which is injected into the RNN recurrence via weight matrix W_{ih}.

The interaction between the two networks proceeds as follows:

- **Visual Feature Extraction (CNN):** The input image is fed into a convolutional network (such as a pre-trained VGG-16). Crucially, the final classification layer (which predicts ImageNet class scores with Softmax) is discarded. Instead, we extract the dense feature vector $v \in \mathbb{R}^{4096}$ from the penultimate fully connected layer (`FC-4096`), capturing the global semantic content of the image.
- **The Recurrence Equation (Slide 118):** While a standard RNN computes $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$, in Image Captioning the visual feature vector v is added as a dedicated third input term:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + W_{ih}v)$$

Note

What Learns vs. What Remembers (Parameters vs. Context):

- **The Learnable Weights (What changes during training):** The matrices W_{xh}, W_{hh}, W_{ih} (alongside output projection W_{hy}) are the actual parameters of the model. During training via BPTT, backpropagation updates **only these weight matrices**. They learn the optimal transition rules: how to interpret words, how to advance memory, and how to project image features. At test time, all W matrices are frozen.
- **The Hidden State (h_t):** The hidden state does **not** contain learnable weights. It is an activation vector acting as temporary working memory (functioning like a continuous Markovian state) that compresses the visual context v and the sequence of words generated so far. It updates dynamically at every time step and resets to $\vec{0}$ for each new image.

- **The Inputs (x_t and v):** Vector v is the static visual embedding anchoring the language to the image, while x_t is the text input ($x_0 =$ <START> at $t = 0$, followed by the word sampled at step $t-1$).

Autoregressive Caption Generation

At inference time, the unrolled RNN generates the descriptive sentence word by word, as illustrated in Figure 3.8.

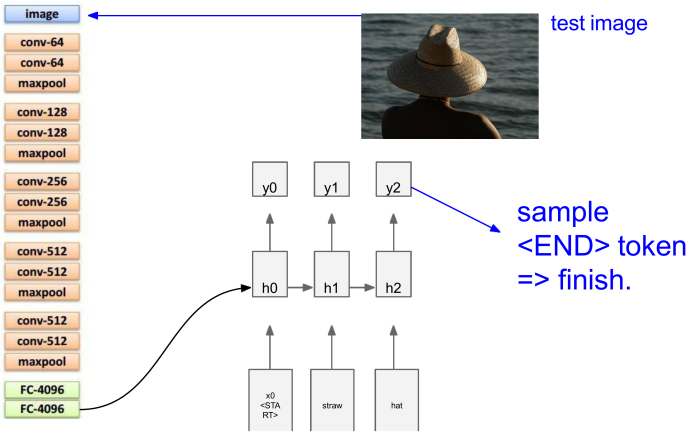


Figure 3.8: Unrolled test-time generation of an image caption (Slide 123). The network generates words sequentially from a starting token until emitting the special <END> token.

At each step, the hidden state h_t projects raw scores (logits) y_t, which are converted into a probability distribution over the word dictionary via Softmax. Generation unfolds sequentially:

1. **Initialization ($t = 0$):** Given visual vector v, initial memory $h_{-1} = \vec{0}$, and start token $x_0 =$ <START>, the network computes h_0 and predicts distribution y_0. The most probable word, *"straw"*, is sampled.
2. **Sequential Steps ($t = 1, 2, \ldots$):** The newly sampled word is fed back as the input for the next time step alongside memory h_t and visual vector v:
   - Feed $x_1 =$ *"straw"* $\longrightarrow$ computes $h_1 \longrightarrow$ samples *"hat"*.
   - Feed $x_2 =$ *"hat"* $\longrightarrow$ computes $h_2 \longrightarrow$ samples <END>.
3. **Stopping Criterion:** Sampling the special token <END> signals that the sentence is complete, terminating the autoregressive loop with final caption *"straw hat"*.

3.4 Backpropagation Through Time and Gradient Flow

Training an RNN requires computing the gradient of the total loss L with respect to the shared weight matrices W. This process is known as **Backpropagation Through Time (BPTT)**.

3.4.1 BPTT and Gradient Flow in Vanilla RNNs

BPTT proceeds in two phases:

1. **Forward Pass:** The input sequence is processed step by step, computing intermediate hidden states h_t and loss terms L_t.
2. **Backward Pass:** Gradients are backpropagated backward in time from step $t = T$ to $t = 1$, accumulating gradient contributions to the shared weights at every step.

When backpropagating from hidden state h_t to previous state h_{t-1}, the gradient is multiplied by the recurrent weight matrix W_{hh}^T at each step (Slides 145–147).

3.4.2 Exploding and Vanishing Gradients

Backpropagating gradients across multiple time steps involves repeated multiplication by W_{hh} (neglecting the non-linearity for simplicity). As illustrated in Figure 3.9, this repeated multiplication leads to two failure modes (Slide 157):

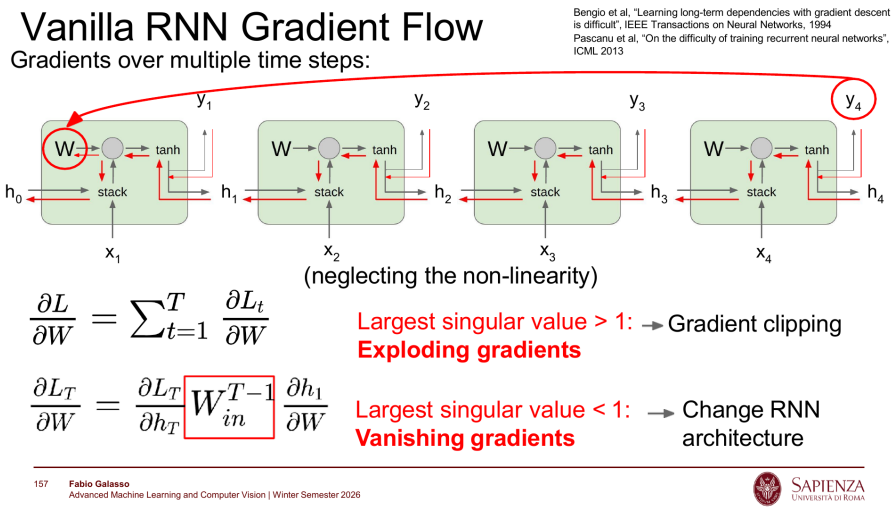


Figure 3.9: Gradient flow across multiple time steps in a Vanilla RNN (Slide 157). Repeated multiplication by W_{hh} inevitably causes either exploding or vanishing gradients.

- **Exploding Gradients (Largest singular value > 1):** Gradients grow exponentially with sequence length, causing numerical instability.
  Solution: *Gradient Clipping*. When the gradient norm $\|g\|$ exceeds a threshold τ, it is clipped:

$$g \longleftarrow \frac{\tau}{\|g\|} g \quad \text{if } \|g\| > \tau$$

- **Vanishing Gradients (Largest singular value < 1):** Gradients decay exponentially toward zero as they travel back in time. Early weights receive negligible updates, making it practically impossible to learn long-term dependencies.
  Solution: Vanishing gradients cannot be fixed by gradient clipping; they require redesigning the architecture with additive interactions $\longrightarrow$ **LSTM**.

3.4.3 Vanilla RNN Trade-offs

The strengths and limitations of Vanilla RNNs can be summarized as follows (Slide 158):

Key Insight

Vanilla RNN Trade-offs (Slide 158):

- **Advantages:**
  - Can process input sequences of arbitrary length.
  - Model parameter size remains strictly constant regardless of input length.
  - Same weights are applied at every time step, providing temporal symmetry.
  - In theory, computation at step t can use information from many steps back.
- **Disadvantages:**
  - Recurrent computation is slow and cannot be parallelized across time.
  - In practice, vanishing gradients make it difficult to access information from many steps back.

3.5 Long Short-Term Memory

To resolve the fatal vanishing gradient problem inherent in Vanilla RNNs, Hochreiter and Schmidhuber (1997) introduced the **Long Short-Term Memory (LSTM)** network.

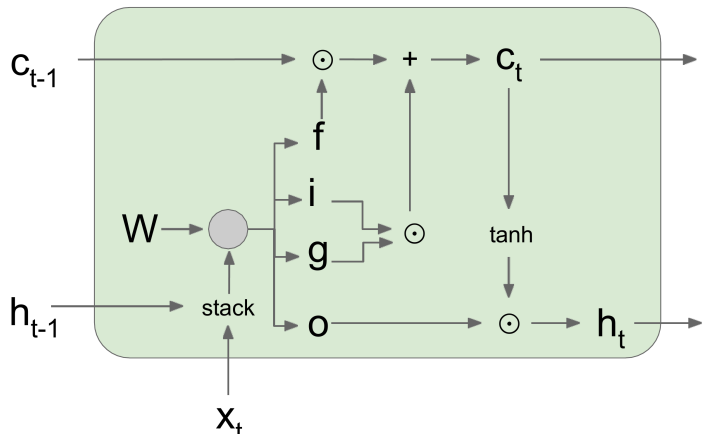
3.5.1 The Dual-State Architecture

While a Vanilla RNN maintains only a single hidden state h_t, an LSTM explicitly decouples memory into two complementary internal state vectors:

1. **Hidden State (h_t):** The short-term working state, used to emit immediate predictions and interface with adjacent layers.
2. **Cell State (c_t):** The long-term persistent memory. It functions as an uninterrupted information highway where memories can flow through time with minimal linear attenuation.

3.5.2 The LSTM Cell Architecture

Figure 3.10 illustrates the internal structure of the LSTM cell and its governing equations.



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Figure 3.10: The LSTM cell diagram and its update equations (from course slides). At time step t, the cell stacks h_{t-1} and x_t to compute four gates (f, i, g, o), updating the cell state c_t additively and producing the hidden state h_t.

3.5.3 Detailed Breakdown of Cell Components

To understand how information flows through the cell in Figure 3.10, we can trace each input, parameter, gate, and intermediate operation:

- **Inputs to the Cell:**
  - x_t: The input vector at the current time step t (e.g., word embedding or extracted visual feature vector).
  - h_{t-1}: The hidden state from the previous time step $t-1$, providing short-term context.
  - c_{t-1}: The cell state from the previous time step $t-1$, carrying long-term memory along the upper horizontal highway.
- **Vector Stacking and Weight Matrix W:**
  - $\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$ (**stack**): Concatenates h_{t-1} and x_t into a single composite vector.
  - W: The learnable weight matrix of the cell. Multiplying W by the stacked input computes all four gate activations simultaneously in a single linear transformation:

$$\begin{pmatrix} f \\ i \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

- **The Four Gating Mechanisms (f, i, g, o):**
  - f (**Forget Gate**, σ): *Whether to erase cell memory*. Uses the Sigmoid activation $\sigma(z) \in [0,1]$. Acting as a soft valve, an activation close to 0 clears that memory dimension, whereas 1 preserves it completely.
  - i (**Input Gate**, σ): *Whether to write to the cell*. Uses Sigmoid $\sigma(z) \in [0,1]$. It acts as a filter deciding which portions of the incoming information are relevant enough to be stored.
  - g (**Gate Gate / Candidate State**, tanh): *What candidate information to write*. Uses $\tanh(z) \in (-1,1)$ to propose new candidate values to be written into the cell state.
  - o (**Output Gate**, σ): *How much of the cell to reveal*. Uses Sigmoid $\sigma(z) \in [0,1]$ to control how much of the internal cell state is exposed to the outside as the working hidden state h_t.
- **Element-Wise Operations and State Updates:**
  - $\odot$ (**Hadamard Product**): Pointwise vector multiplication used for gating. Multiplying by a $[0,1]$ gate value smoothly scales each dimension.
  - $f \odot c_{t-1}$: Forget gate modulation. It selectively suppresses outdated memories from the previous cell state.
  - $i \odot g$: Input gate modulation. It scales the candidate information g by the input gate i, letting only selected features enter the memory.
  - $+$ (**Additive Interaction**): Merges the preserved past memory with the new candidate memory:

$$c_t = f \odot c_{t-1} + i \odot g$$

    This addition is the core innovation of the LSTM: it bypasses the repeated multiplicative weight transformations of vanilla RNNs.
  - $\tanh(c_t)$: Squashes the accumulated cell state values into $(-1,1)$ before filtering for output.
  - $h_t = o \odot \tanh(c_t)$: Produces the current hidden state h_t, which is both routed to the next time step $t+1$

and emitted upwards to produce predictions or feed higher layers.

3.5.4 Why LSTM Solves the Vanishing Gradient Problem

The fundamental reason LSTMs succeed where Vanilla RNNs fail lies in the gradient flow along the cell state highway, illustrated in Figure 3.11.

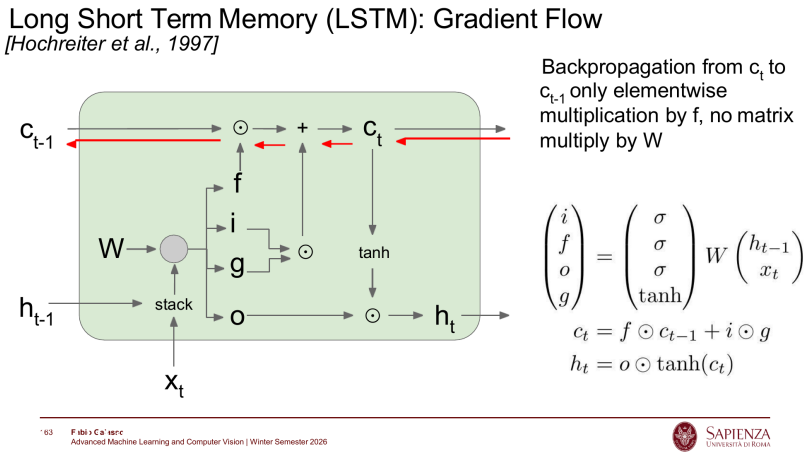


Figure 3.11: Uninterrupted gradient flow along the cell state highway in an LSTM (from course slides). Backpropagation from c_t to c_{t-1} involves only element-wise multiplication by f, avoiding matrix multiplications by W.

Key Insight

The Uninterrupted Gradient Highway: Why LSTM Overcomes Vanishing Gradients

In a Vanilla RNN, backpropagating through time repeatedly multiplies error gradients by the recurrent weight matrix W_{hh}^T and saturating tanh derivatives, causing the signal to decay exponentially towards zero ($W_{hh}^T \cdot W_{hh}^T \cdots \to 0$).

In an LSTM, backpropagating from c_t to c_{t-1} along the upper cell state highway yields $\frac{\partial c_t}{\partial c_{t-1}} = f$:

- **No Weight Matrix Multiplications:** Backpropagation along the highway involves only an element-wise multiplication by the forget gate vector f, with **no matrix multiplication by** W. There is no risk of exponential degradation caused by repeated matrix powers.
- **Selective Memory Retention ($f \approx 1$):** The forget gate outputs values in $[0,1]$. If the network learns that an early piece of context is critical and sets $f \approx 1$, the local gradient derivative is practically 1.
- **Lossless Flow Across Hundreds of Steps:** Multiplying by 1 repeatedly preserves the error magnitude:

$$1 \times 1 \times \cdots \times 1 = 1$$

  The gradient signal flows backwards through dozens or hundreds of time steps virtually intact!
- **The ResNet Analogy:** Just as ResNets use additive identity skip connections ($x + F(x)$) to bypass deep spatial layers, LSTMs use an additive cell state ($f \odot c_{t-1} + i \odot g$) to bypass long temporal sequences.

3.6 Chapter Summary and Conceptual Graph

The concepts developed in this chapter trace the evolution of sequence models from parallel temporal convolutions to recurrent memory networks and additive gradient highways:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** Handling variable-length sequences with temporal ordering prompts using 1D convolutions; enforcing causality prevents future lookahead, while dilation expands context without sacrificing parallel training.
- **From Node 2 to 3:** Despite their parallel speed, TCNs have a fixed receptive field buffer; Recurrent Neural Networks introduce true dynamic memory via an evolving hidden state h_t with parameter sharing.
- **From Node 3 to 4:** Training recurrent graphs requires unrolling them through time and calculating loss gradients backwards across all time steps via Backpropagation Through Time (BPTT).
- **From Node 4 to 5:** Unrolling across long horizons exposes BPTT to repeated multiplications by W_{hh}^{T-1}. If the spectral radius exceeds 1, gradients explode (controlled via Gradient Clipping); if less than 1, gradients vanish exponentially.
- **From Node 5 to 6:** Because vanishing gradients represent an architectural failure of Vanilla RNNs, Hochreiter and Schmidhuber redesigned the memory cell: separating working state h_t from a dedicated cell state c_t.
- **From Node 6 to 7:** Regulating the cell state via input, forget, and output gates (i, f, o) and applying purely additive updates $(+)$ establishes an uninterrupted gradient highway, allowing LSTMs to preserve long-range dependencies across hundreds of steps.

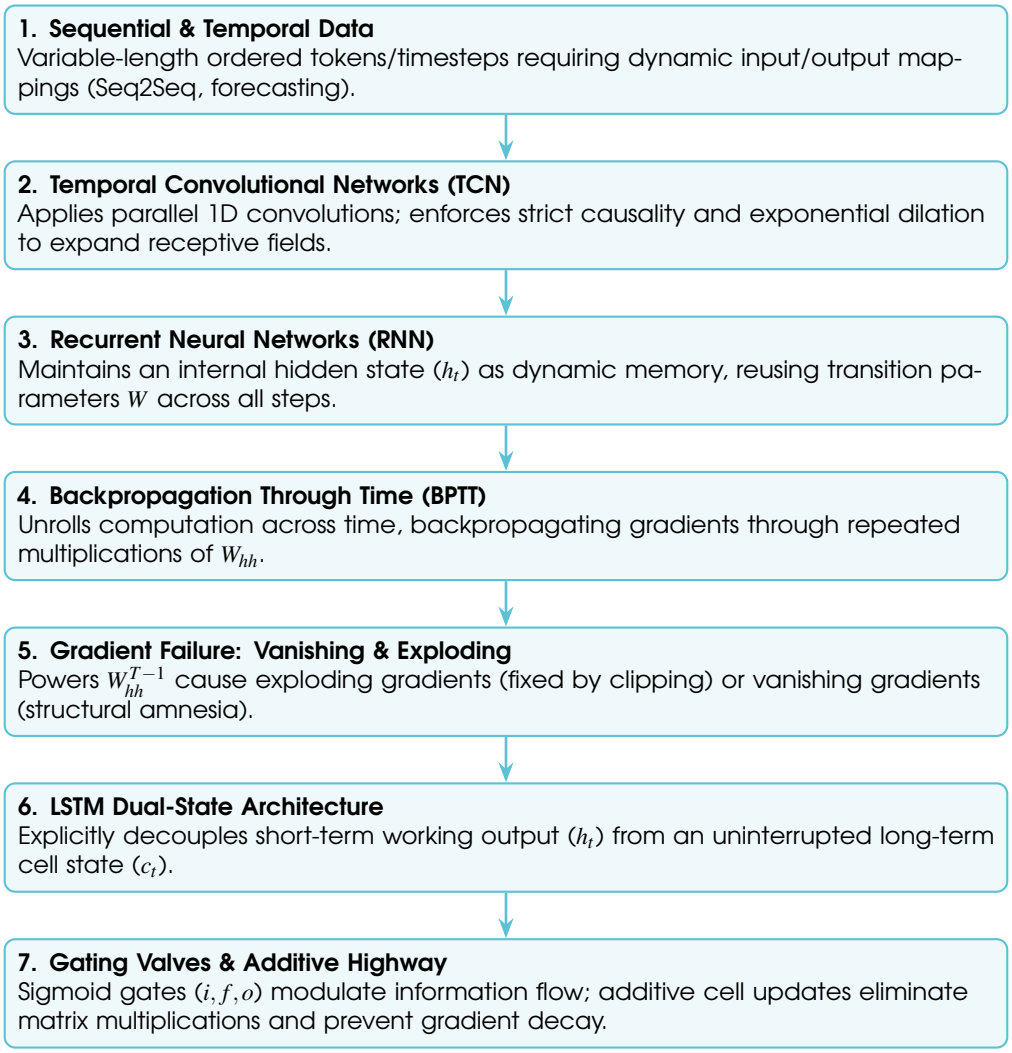


Figure 3.12: Conceptual roadmap of sequence modeling, recurrent dynamics, and LSTMs.

Bibliography

[1] Fabio Galasso. *Recap on Image Classification and Neural Networks*. Advanced Machine Learning Course Slides, Sapienza Università di Roma. Sept. 2025.